

Self Introduction

Hi,

As you know my name is ABCD I am having 4.2 years of experience in IT industry, in that 4years as ETL tester and remaining as manual tester.

Recently I have worked in HCL Technologies, Bengaluru and worked on Menards Retail Domain project, the client name is Menard data warehouse from United States.

In that project,

- I have used Oracle as Database,
- SQL as to write queries and to validate data,
- Informatica as ETL Tool,
- HP ALM as Project Management Tool (Bug Tracking Tool). (Application Lifecycle Management)
- TOAD is used as data accessing.
- I am very good to write SQL queries.
- I am a Quick learner, Team Player,
- Willing to learn new technologies.

And this is about my-self introduction.

Role: The Role in my last Project as Software Testing Engineer.

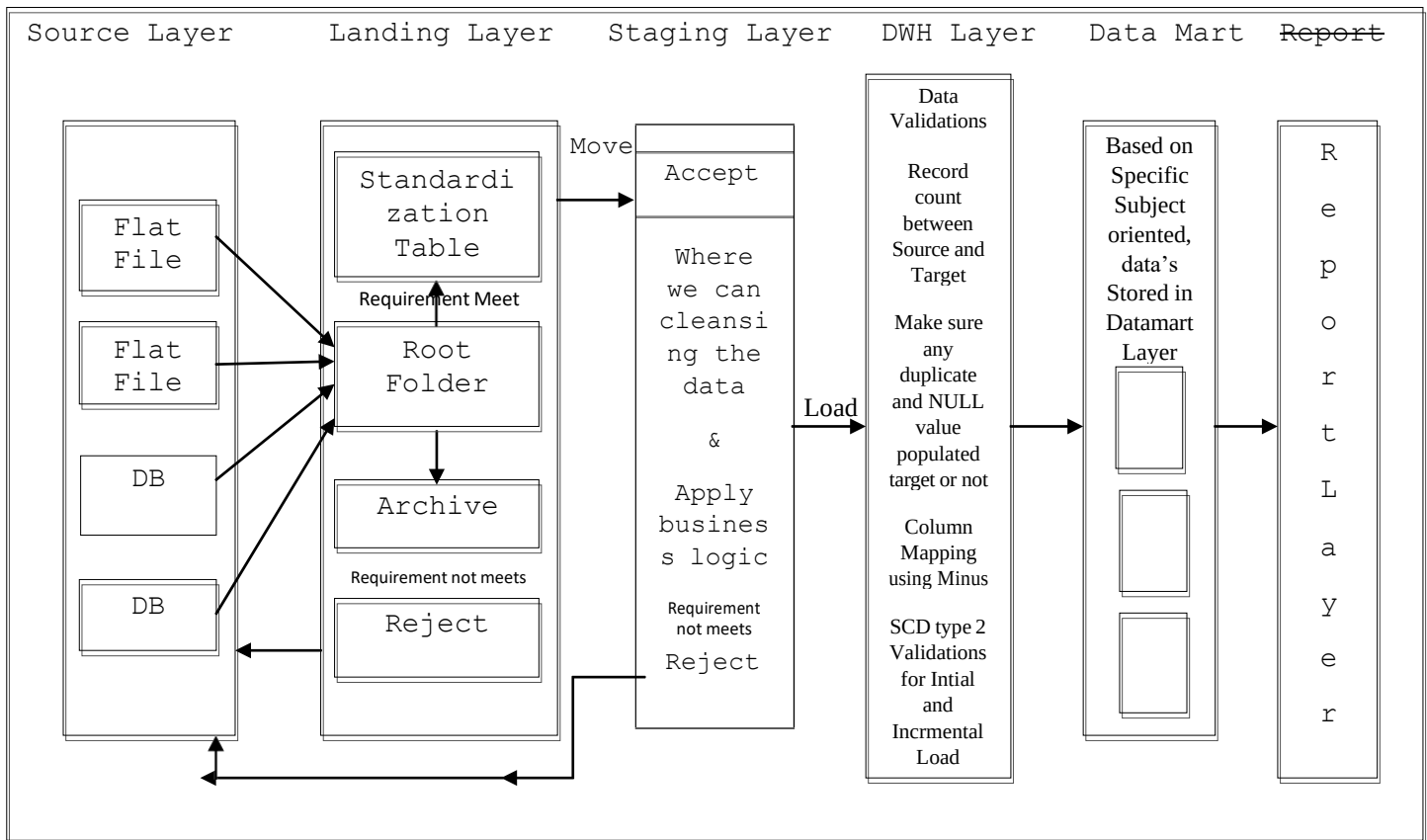
And responsibilities are:

- First I go through the technical documents such as source target mapping (STM).
- Then I have understood the client requirement and come up with high level scenario.
- Then I have developed the test cases and those test cases are undergone peer review.
- Once the test cases approved by Team Lead, we have uploaded in HP ALM.
- Once build deployed in testing environment, we start executing test cases.
- If any deviation between actual and expected result then I have raise the bug report to particular developer and I have follow-up until to it has resolved.
- I also assist team lead to produce test plans.
- Even I have part of all calls like scrum call, client call, Standup call, weekly call and monthly call.
- At end of the day (EOD), I have submitted the daily status report to team lead.

And my Daily Activities are:

- Once I login, I go through the outlook if any response required I will reply.
- I involve in stand-up call where team members update ourselves regarding project status like what I did yesterday and what I have to do today and is there any obstacles.
- Then I carry test activities.
- I also involved in scrum calls and client calls.
- At the end of the day (EOD), I have submitted the daily status report to team lead

Architecture of Project/ how data flow in your Project/ Stages of Project



- First I speak about my project.
- My project name is Menard's data warehouse client is from United States.
- The company sells various numbers of products. Menards has various stores in United States.
- They have massive network connection through retailers, stockiest etc.

In that project we have 1) Source Layer 2) landing Layer 3) Staging Layer 4) Data warehouse Layer 5) Data Mart Layer.

- Here we have heterogeneous sources some are in flat file and DB. (A Flat File database is also known as the text database.)
- Once it is landed in to root folder of landing area, from that point actual ETL mechanism is going to start to check the naming convention of file, time stamp and perform Meta-Data validation for name of attribute, data type, size and constraints.
- Once it meet the all the requirement then it moves to the standardization table, at the same time one copy send to archive folder for future references (Up to 30 days).
- If it is not meet our requirements then the data's are move to reject folder, from that we sent back to respective sources with error description by email triggering system.
- Whatever data present in the standardization table it move to the staging layer where we can cleansing the data and apply the business logic. (Business Rules)

- Suppose if requirement not meet then those are sent back to respective sources with error description through email trigger system.
- Now data are loaded in to data warehouse, where we can perform the data validations such as
 1. Compare the record count between source and target,
 2. Make ensure that any duplicate values populated in target or not
 3. Make ensure that any NULL values populated in target or not.
 4. We perform column mapping using minus query, we make sure that the expected data loaded in to target as per requirement. Then we expect the zero rows return.
- And also done the SCD type 2 validation for initial and incremental load.
- And also validate that data's according to business logics.
- Based on specific subject oriented, data's are stored separately in data mart layer.

What is a defect?

The variation between the actual results and expected results is known as defect.

If a developer finds an issue and corrects it by himself in the development phase then it's called a defect.

What is a bug?

If testers find any mismatch in the application/system in testing phase then they call it as Bug.

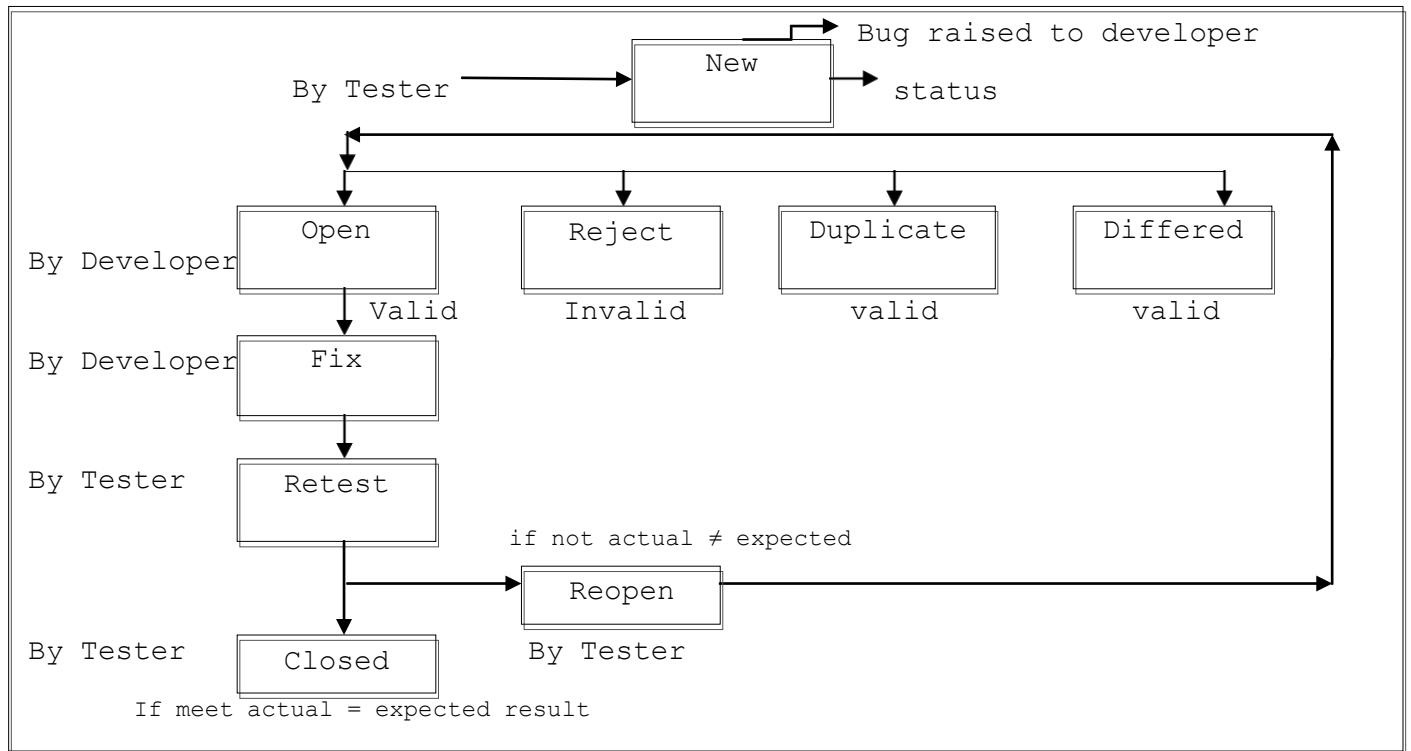
What is an error?

We can't compile or run a program due to coding mistake in a program. If a developer unable to successfully compile or run a program then they call it as an **error**.

What is a failure?

Once the product is deployed and customers find any issues then they call the product as a failure product. After release, if an end user finds an issue then that particular issue is called as **failure**

Defect life cycle or Bug life cycle.



- During the execution of the test cases if we found any deviation between actual and expected result, then I have taken artefacts and evidence and contact onsite co Ordinator and team (developer + tester). Based on that call discussion I have raised a bug report to a particular developer with status as **NEW**.
- Now developer may undergone four different scenario
 - a) Open b) reject c) duplicate d) differed
- a) **Open:** In which whatever I have reported it is valid then he maintain a status as open it means he working on that. Then
- b) **Reject:** He maintain status as reject when whatever I reported is invalid. Then
- c) **Duplicate:** He maintain status as duplicate when whatever I reported is valid but it is already reported by other tester of the team. Then
- d) **Differed:** He maintains status as differed when whatever I reported is valid but they not consider in current sprint release they may consider in future release.
- After completing the build by developer he deployed in testing environment and maintains status as **Fix**.
- Then tester maintains a status as **Retest** where tester will test the build.
- Then tester maintains status as **Closed** when the actual and expected result matches.
- If the result is not matches then tester mention a status as **reopen** to the particular developer again.
- Till it repeated to reach the status as **closed**.

Data Warehouse:

It is a repository or place in which we can store the historical data, by which generate the reports and analysis the business key metric fields in order to improve the business.

The **Characteristics** of data warehouse are

It is subject oriented: in which we can focus on the particular area of analysis.

Integrity: by which combine multiple data from multiple sources.

It is time variant: means we can store historical data.

It is non-volatile, means we cannot modify any data within it.

If suppose want to change then added as new record.

Normal Database (OLTP)	Data Warehouse (OLAP)
OLTP (Online Transaction Processing)	OLAP (Online Analysis Process)
• It records the user current transaction data	• It maintain the historical data
• Tables and Joins are complex since they are <u>normalized</u>. (Avoid duplicates)	• Tables and Joins are simple since they are de-normalized.
• Optimized for keeping small amount of data	• Optimized for keeping large amount of data
• MB to GB	• GB to TB
• Small to Large	• Large to Very Large
• Entity relationship Modelling technique is used	• Dimension modelling technique is used
• Optimized (used) for write operations like insert, update, Delete	• Primarily Read only operation
• Application oriented	• Subject oriented
• Only single stream	• Integrity
• Volatile	• Non-volatile
• Handling single record at a time	• Handling multiple records at a time

Normalization:

It is the process of splitting the table into another table in order to minimize the redundancy.

- Redundancy data means repeated or duplicate data.
- Due to the redundancy the problems arise are.
 1. Occupied more space in DB.
 2. Anomaly in insert, update, delete

Types of Normal Forms

1. 1NF, 2. 2NF, 3. 3NF, 4. BCNF or 3.5NF (BOYCECODD)

1. 1NF: 1st Normal Form:

Suppose data stored in a table

- That would be a single value in attribute.
- In the table column attributes are unique.
- The storing of data may be in any order.
- No duplicates (Similar type of data)

Bad Data: Suppose data not followed 1NF that data called Bad Data

2. 2NF: 2nd Normal Form:

- Suppose data stored in a table it should be 1NF
- It eliminates **partial** dependency.
- Partial Dependency occurs when a non-prime attribute is functionally dependent on part of a candidate key.
- A functional dependency $X \rightarrow Y$ is a partial dependency if Y is functionally dependent on X and Y cannot be determined by any proper subset of X.
- For example, we have a relationship $AC \rightarrow B$, $A \rightarrow D$ and $D \rightarrow B$...

3. 3NF: 3rd Normal Form:

- It should be in the Second Normal form.
- Its eliminate Transitive Dependency.
- What is Transitive Dependency?
- When an indirect relationship causes functional dependency it is called Transitive Dependency.
- If $P \rightarrow Q$ and $Q \rightarrow R$ is true, then $P \rightarrow R$ is a transitive dependency.

4. Boyce Codd Normal Form or 3.5 Normal Form

- It should be in the Third Normal Form.
- And, for any dependency $A \rightarrow B$, A should be a super key.
- The second point sounds a bit tricky, right? In simple words, it means, that for a dependency $A \rightarrow B$, A cannot be a non-prime attribute, if B is a prime attribute.

Dimension Table:

In data warehouse we store the data in two forms 1. Dimension table 2. Fact table.

Dimension Table:

- It is detail descriptive data about the business.
- It contains Primary Key (PK)
- It contains Texture data

Types of Dimension Table

1. SCD (Slowly Changing Dimension)
2. Conformed Dimension Table
3. Junk Dimension Table
4. Degenerative Dimension Table
5. Role-play Dimension Table

1. SCD (Slowly Changing Dimension)

The values or data's of the table are changes over a long period hence we call as slowly changing dimension.

There three types of SCD

a. SCD1, b. SCD2, c. SCD3

- a. **SCD type 1:** In which we can maintain only current data. This can be achieved by overwrite on old data.
- b. **SCD type 2:** In which we can maintain current and all or complete historical data. This can be achieved by inserting as new row.
- c. **SCD type 3:** In which we can maintain current and partial historical data. This can be achieved by inserting as pair of attribute.

2. Conformed Dimension Table

It is the dimension table that can be shared by multiple fact tables. **Or**

The dimension table which is having same meaning to all fact tables.

Ex: Time dimension Table (Dim_time)

3. Junk Dimension Table

These are the unwanted data's attribute in a fact table. These are nothing but the collection of transactional code flag which are not same as other dimension table.

Ex: Mode of Payment in which

Cash
Cheque
Credit

Ex: Store in which

Market Place
Warehouse

Ex: Customer assist (review) in which

Yes
No

4. Degenerative Dimension Table

In which there is no physical separate dimension table available, but the key (PK) is available in fact table.

Ex: Purchase Order Number, Policy Number, Order Number

5. Role-play Dimension Table

A dimension table is having unique label for all business aspects in a same fact table. **or**

A dimension table rapidly used for all attributes with unique label.

Ex: Online Purchase in which the date dimension

Order date, payment date, packing date, shipping date and delivery date.

In above all attribute the date is unique (No duplicate) so that **time dimension table is a role-play dimension table.**

Fact Table:

- These are nothing but the fact or measures or summarized data about the business.

- It is having foreign key.
- It contains numeric values.

Types of Fact Table:

1. Additive Fact Table:

- In which the fact value is generated by considering all dimension tables.
- Ex: sales

2. Semi – Additive Fact Table:

- In which fact value is generated by considering few dimension tables.
- Ex: quantity in hand (balance quantity), bank balance.

3. Non – Additive Fact Table:

- In which fact value is generated without considering any dimension table.
- Ex: sales tax, unit price.

4. Fact-Less Fact Table:

- It does not contain any fact or measure.
- It contains foreign keys.

Difference between Dimension Table and Fact Table:

Dimension Table	Fact Table
• It contains details descriptive about the business	• It contains measures about the business
• It is having primary key	• It is having foreign key
• It contains textual data	• It contains measures in numeric
• In OLTP dimension table are normalized • In OLAP dimension table are de-normalized	• It is always de-normalized
• Dimension table contains less data	• Fact table contains more data

Schema:

Schema is the skeleton structure that represents the logical view of the entire database. It defines how the data is organized and how the relations among them are associated.

Types of Schema:

1. Star Schema:

- A centralized located fact table which surrounded by multiple dimension table..
- It looks like a star hence it is called star schema.

2. Snow Flake Schema:

- It is the extension of star schema, in which dimension table exploded into another dimension table.

Difference between Star Schema and Snow Flake Schema:

Star Schema	Snow Flake Schema
• A centralized located fact table which surrounded by multiple dimension table.	• It is the extension star schema, in which dimension table exploded into another dimension table
• Dimension table are de-normalized	• Dimension table are normalized
• Simple in structure hence joins and query are simple	• Complex or complicated in structure hence joins and tables are complicated
• Execution of the SQL query is high Performance	• Performance of the query is less

SQL (STRUCTURAL QUERY LANGUAGE)

- SQL is a standard query language for storing, manipulating and retrieving data in databases.
- It is a command based language
- Each command having its own meaning.
- By this language we can communicate with DB or DWH.
- It is non-case sensitive.

Commands of SQL or Sub-languages of SQL

DDL (Data Definition Language)	Create	Add Modify Drop Rename
	Alter	
	Drop	
	Rename	
	Purge	
DML (Data Manipulation Language)	Flash Back	
	Truncate	
	Insert	
	Update	
	Delete	
DCL (Data Control Language)	Select	
	Grant:	
	Revoke:	
TCL (Transaction Control Languages)	Rollback	
	Commit	
DQL (Data Query Language)	Select	

Data Types in SQL

Number	:	It accept only numerical value
Char	:	It accept the character and numerical value
Varchar2	:	It accept the character and numerical value
Date	:	It accept the dates

CHAR	VARCHAR2
Fixed Length	Flexible Length
Name char(10)	Name varchar2(10)
Raju----- ('-' empty is unused)	Raju -----('-' send back to DB)
Maximum size of length is 0 - 255	Maximum size of length is 0 - 255

1. DDL (Data Definition Language):

- DDL commands are used to define the DB objects.
- These commands directly interact with DB hence auto commit (Automatic Save).
- These commands are not rollback.
- These commands deal with only structure of the table.

CREATE: This command is used to define or create the DB object like table, views, Store procedure etc.

Syntax:

```
CREATE TABLE <TABLE NAME> (COLUMN_NAME DATATYPE (SIZE),  
COLUMN_NAME2 DATATYPE (SIZE), COLUMN_NAME3 DATATYPE (SIZE).....);
```

Example:

```
CREATE TABLE COLLEGE (REGNUMBER VARCHAR2 (10), STUDENTNAME CHAR  
(30), PHONENUMBER NUMBER (10), BRANCH VARCHAR2 (10));
```

DESC:

Display the description of the table (structure of table)

Example: DESC College;

REGNUMBER	VARCHAR2 (10)
STUDENTNAME	CHAR (30)
PHONENUMBER	NUMBER (10)
BRANCH	VARCHAR2 (10)

ALTER: This command is used for changes or modify structure of the attribute i.e. this command is meant for only structure of the attribute.

(Attribute name datatype (size))

❖ **ADD:**

By using this command we can add or insert the attribute (column) to the existing table.

Syntax:

```
ALTER TABLE <TN> ADD (CN1 DATATYPE (SIZE), CN2 DATATYPE (SIZE), .....);
```

Example:

```
ALTER TABLE COLLAGE ADD (HOD VARCHAR2 (10), CITY VARCHAR2 (20));
```

❖ **MODIFY:**

By using this command we can change the data type and size.

Syntax:

```
ALTER TABLE <TABLENAME> MODIFY (COLUMN_NAME1 DATATYPE  
(SIZE), COLUMN_NAME2 DATATYPE (SIZE));
```

Example:

```
ALTER TABLE COLLAGE MODIFY (HOD CHAR (10));
```

❖ **DROP:**

By using this command we can delete the column permanently from the DB.

Syntax:

ALTER TABLE <TABLENAME> DROP (COLUMN_NAME1);

Example:

ALTER TABLE COLLAGE DROP (CITY);

❖ **RENAME:**

By using this command we can give the new name to the existing name to attribute (rename).

Syntax:

**ALTER TABLE <TABLENAME> RENAME COLUMN EXISTING
COLUMN_NAME TO NEW COLUMN_NAME;**

Example:

ALTER TABLE COLLAGE RENAME COLUMN HOD TO BRANCHHEAD;

DROP:

By using this command we delete the table permanently from DB.

Syntax:

DROP TABLE <TABLE NAME>;

Example:

DROP TABLE COLLAGE;

RENAME

- By using this command we can give new name to the table.

Syntax:

RENAME <EXISTING TABLE_NAME> TO <NEW TABLE_NAME>;

Example:

RENAME COLLAGE TO COLLAGE_1;

PURGE:

By using this command we remove DB objects permanently from recycle bin.

Syntax:

PURGE TABLE <TABLENAME>

OR

DROP TABLE <TABLENAME> PURGE

FLASK BACK:

This command is used for restoring the DB objects from recycle bin.

Syntax:

FLASH BACK <DB OBJECT NAME> TO BEFORE DROP;

Example:

FLASHBACK TABLE COLLAGE TO BEFORE DROP;

To view the objects in recycle bin:

SELECT * FROM RECYCLE BIN

or

SHOW RECYCLE BIN.

TRUNCATE

- By using this command we can delete whole data of the table at a single shot, but structure remains same.

Syntax:

TRUNCATE TABLE <TABLE_NAME>

Example: **TRUNCATE TABLE COLLAGE_1;**

DROP	TRUNCATE
We delete the table in a single shot permanently from DB	We delete the table records in a single shot permanently from DB but structure remains same.
Performance is high	Performance is low

2. DML (Data Manipulation Language):

- DML commands are interact with the DB through buffer hence these are not auto commit(save).
- These commands can roll back.
- By using these commands we can manipulate or organize the data in a table.

INSERT

By using insert command we can insert or place the data into a table.

Syntax:

a. INSERT INTO <TABLE_NAME> VALUES ('VALUE1', 'VALUE2','VALUE3'.....);

Example:

INSERT INTO COLLAGE_1 VALUES ('1', 'RAVI', 'ME');

b. INSERT INTO <TABLE_NAME> (CN1, CN2, CN3....) VALUES ('VALUE1', 'VALUE2', 'VALUE3'.....);

Example:

```
INSERT INTO COLLAGE_1 (SLNO, NAME, BRANCH) VALUES ('1', 'RAVI', 'ME');
```

c. Insert multiple values

```
INSERT INTO TABLE <TABLE_NAME> VALUES ('&CN1', '&CN2', '&CN3',..... );
```

Example:

```
INSERT INTO TABLE COLLAGE_1 VALUES('&SLNO', '&NAME', '&BRANCH',... .);
```

UPDATE

By using this update command we can modify the existing data in the table.

Syntax:

a. UPDATE <TABLE_NAME> SET COLUMN_NAME = 'UPDATE VALUE'

Example:

```
UPDATE COLLAGE_1 SET BRANCH= 'COMP SCI';
```

b. UPDATE <TABLE_NAME> SET COLUMN_NAME= ' UPDATE VALUE' WHERE <CONDITION>;

Example:

```
UPDATE COLLAGE_1 SET BRANCH= ' COMP SCI' WHERE NAME= RAVI;
```

DELETE

By using this delete command we can delete the row from table.

Syntax:

a. Delete the specific row using where clause

```
DELETE FROM <TABLE_NAME> WHERE <CONDITION>;
```

Example:

```
DELETE FROM COLLAGE_1 WHERE SLNO=4;
```

b. Completely delete the table:

```
DELETE FROM <TABLE_NAME>
```

Example: DELETE FROM COLLAGE_1;

SELECT:

By using this select command or clause we can retrieve or fetch or pull the data from data ware house or DB.

Syntax:

SELECT * FROM <TABLE_NAME>;

Example:

SELECT * FROM COLLAGE_1;

Note: here * indicates all attributes data. (i.e., complete table data)

DELETE	TRUNCATE
Similarities: In these both commands we delete data from the table but structure remains same.	
• We delete the specific row or whole data from the table.	• We delete all the data from the table at a single shot.
• By using where clause we delete specific row	• Truncate doesn't support the where clause.
• Performance is low	• Performance is high
• It is a DML command	• It is DDL command
• These are not auto commit	• These are auto commit
• Can be roll-back	• Cannot be roll-back

Order of SQL Query or Structure of SQL or Hierarchy of Clauses or Order of writing SQL Query:

SELECT : By that we can retrieve the data (i.e., what we want to specify)
DISTINCT : By using this we can get (retrieve) the unique record.
FROM : Here we can specify the table name.
WHERE : Here we filter the general data by specifying the conditions.
GROUP BY : Here we club the similar kind of data based on column.
HAVING : Here we filter the aggregate data by specifying the condition.
ORDER BY : Here we sort the result set in ascending or descending order, by default ascending

Note: Here **SELECT** and **FROM** clause are mandatory and other are optional.

Select: Syntax

SELECT COLUMN_NAME1, COLUMN_NAME2... FROM <TABLE_NAME>;

Example:

SELECT EMPNO, ENAME, JOB FROM SCOTT.EMP;

Distinct: Syntax

SELECT DISTINCT <COLUMN_NAME> FROM <TABLE_NAME>;

Example: DISTINCT DEPTNO FROM SCOTT.EMP;

From: Syntax

SELECT * FROM SCOTT.EMP;

Where: Syntax

Display the records of employees those are belongs to 'deptno 30'

SELECT * FROM SCOTT.EMP WHERE DEPTNO=30;

Display the name of the employees those who are getting salary > 2000

SELECT NAME FROM SCOTT.EMP WHERE SAL>2000;

Display the employees name and salary those designation as clerk

SELECT ENAME, SAL FROM SCOTT.EMP WHERE JOB='CLERK';

Relational Operations:

("=", ">", "<", "!=", "<>", "=>", "<=")

Logical operations:

(AND, OR).

Display the records of emp those are belong to dept no 30 and salary > 2000

SELECT * FROM SCOTT.EMP WHERE DEPTNO = 30 AND SAL > 2000;

Display the records of the employee those are belong to dept no 30 or salary > 2000

SELECT * FROM SCOTT.EMP WHERE DEPTNO=30 OR SAL>2000;

AGGREGATE FUNCTIONS

These are the functions which act on multiple rows to return a single value.

These are works on the group of similar kind of data.

- SUM(COLUMN_NAME);
- COUNT(COLUMN_NAME);
- MAX(COLUMN_NAME);
- MIN(COLUMN_NAME);
- AVG(COLUMN_NAME);

Group By:

It groups the data or field.

Note: In group by function we have to observe (notice) that as below

- The number of columns used in select statement.
- The same number of columns are used in group by clause.

Syntax:

```
SELECT DEPTNO, SUM(SAL) FROM SCOTT.EMP GROUP BY DEPTNO;
```

Having:

- Having is also filter for all aggregate values.
- Check the condition from the query.

Syntax:

```
SELECT DEPTNO, SUM(SAL) FROM SCOTT.EMP GROUP BY DEPTNO HAVING  
SUM(SAL)>9000;
```

```
SELECT DEPTNO, SUM(SAL) FROM SCOTT.EMP WHERE DEPTNO!=10 GROUP BY  
DEPTNO HAVING SUM(SAL)>9000;
```

Order By:

- It orders the results in set.

Syntax:

```
SELECT DEPTNO, SUM(SAL) FROM SCOTT.EMP WHERE DEPTNO!=10 GROUP BY  
DEPTNO HAVING SUM(SAL)>9000 ORDER BY SUM(SAL) ASC;
```

How it works or how it compiles or how it will execute or in what order it execute.

- **From** : It contains table name.
- **Where** : It is used to filtering the data based on condition
- **Group By** : We can club together or combine the similar data based on attribute.
- **Having** : it is used for filtering the data from group by Clause
- **Select & Distinct:** It selects the data to display.
- **Order By** : It used to sort the data in ascending or descending order.

3. DCL (Data Control Language):

DCL commands are used to grant and take back authority from any database user.

Here are some commands that come under DCL:

- Grant
- Revoke

GRANT: It is used to give user access privileges to a database.

Example

```
GRANT SELECT, UPDATE ON MY_TABLE TO SOME_USER, ANOTHER_USER;
```

REVOKE: It is used to take back permissions from the user.

Example

```
REVOKE SELECT, UPDATE ON MY_TABLE FROM USER1, USER2;
```

4. TCL (Transaction Control Language)

TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only.

These operations are automatically committed in the database that's why they cannot be used while creating tables or dropping them.

Here are some commands that come under TCL:

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT: Commit command is used to save all the transactions to the database.

Syntax:

```
COMMIT;
```

Example:

```
DELETE FROM CUSTOMERS
```

```
WHERE AGE = 25;  
COMMIT;
```

ROLLBACK: Rollback command is used to undo transactions that have not already been saved to the database.

Syntax: ROLLBACK;

Example:

```
DELETE FROM CUSTOMERS  
WHERE AGE = 25;  
ROLLBACK;
```

5. DQL (Data Query Language)

DQL is used to fetch the data from the database.

It uses only one command:

- SELECT

SELECT: This is the same as the projection operation of relational algebra. It is used to select the attribute based on the condition described by WHERE clause.

Syntax:

```
SELECT expressions  
FROM TABLES  
WHERE conditions;
```

For example:

```
SELECT emp_name  
FROM employee  
WHERE age > 20;
```

NULL FUNCTIONS

NULL indicate that a data value does not exist in the database.

NVL, NVL2, NULLIF, COALESCE

These above functions deal with NULL.

1. NVL:

We can replace the NULL value by using some value.

Syntax: SELECT NVL('CN/EXPRESSION', 'REPLACE VALUE') FROM TN;

Example: SELECT NVL(COMM, '100') AS COMMISSION FROM EMP;

It fetches the single column.

Example: SELECT EMP.*, NVL(COMM, '100') AS COMMISSION FROM EMP;

It fetches the all column.

Example: SELECT NVL(COMM, 'NO') FROM EMP;

It shows the invalid number.

Example: SELECT NVL(' ', 'NO') FROM DUAL;

2. NVL2:

By NVL2 function we can replace non null value and null value by some value

Syntax: SELECT NVL2('CN/EXPRESSION', 'NOTNULL VLAUE', 'NULL VLAUE');

Example: SELECT EMP.*, NVL2(COMM, '5000', '2000') FROM EMP;

These are two functions deals with NULL

NVL	NVL2
Replace only null column	Replace both not null and null column
Ex: (comm, '5000') null	Ex. (comm., '5000', '2000') not null null
It passes two parameters	It passes three parameters

3. NULLIF: It is nothing but equal comparator.

NULLIF function compares *expression1* and *expression2*. If *expression1* and *expression2* are equal, then NULLIF function returns NULL. Otherwise, it returns the first expression which is *expression1*.

Syntax: NULLIF (Expression1, Expression2);

Example: SELECT NULLIF(12, 12);

Result : NULL (returns null because values are same)

Example. SELECT NULLIF(12, 45);

Result : 12 (returns first value because values are different.)

4. COALESCE: The COALESCE function returns the first non-NULL value from a series of expressions

By using coalesce we can use the n' number of expression for null values.

OR

It is used for N number of null value expression, when first expression is null it retrieves next expression value.

Syntax: SELECT COALESCE (VAL1, VAL2... VAL_N) FROM TN.

Example: SELECT COALESCE(NULL, 1, 2, 3,4);

Result: 1

Example: SELECT COALESCE(1, NULL,2, 3,4);

Result: 2

Example: SELECT COALESCE(1, 2, 3, NULL,4);

Result: 4

Example: SELECT COALESCE(1, 2, 3, 4,NULL);

Result: NULL

Example: SELECT EMP.*,COALESCE(COMM,DEPTNO) FROM EMP;

SQL | Character Functions

Character functions accept character inputs and can return either characters or number values as output.

SQL provides a rich set of character functions that allow you to get information about strings and modify the contents of those strings in multiple ways. Character functions are of the following two types:

1. Case-Manipulative Functions (LOWER, UPPER and INITCAP)

2. Character-Manipulative Functions (CONCAT, LENGTH, SUBSTR, INSTR, REPLACE, TRANSLATE)

UPPER : Converts all letters in the string to uppercase.

Syntax:

UPPER(SQL course)

Input1: SELECT UPPER('geeksforgeeks') FROM DUAL;

Output1: GEEKSFORGEEKS

Input2: SELECT UPPER('dbms\$508%7') FROM DUAL;

Output2: DBMS\$508%7

LOWER : Converts all letters to lowercase.

Syntax:

LOWER(SQL course)

Input1: SELECT LOWER('GEEKSFORGEEKS') FROM DUAL;

Output1: geeksforgeeks

Input2: SELECT LOWER('DATABASE@456') FROM DUAL;

Output2: database@456

INITCAP : Sets the first letter of each word to uppercase. All other letters are set to lowercase.

Syntax:

INITCAP(SQL course)

Input1: SELECT INITCAP('geeksforgeeks is a computer science portal for geeks') FROM DUAL;

Output1: Geeksforgeeks Is A Computer Science Portal For Geeks

Input2: SELECT INITCAP('PRACTICE_CODING_FOR_EFFICIENCY') FROM DUAL;

Output2: Practice_Coding_For_Efficiency

REPLACE : By using replace function we can replace a part of string by another string.

Syntax:

REPLACE(Text, search_string, replacement_string)

Input1: SELECT REPLACE('DATA MANAGEMENT', 'DATA','DATABASE') FROM DUAL;

Output1: DATABASE MANAGEMENT

Input2: SELECT REPLACE ('abcdeabcccabdddeebcc', 'abc') FROM DUAL;

Output2: deccabdddeec

SUBSTR : We can retrieve the part of the string from a string.

Syntax:

SUBSTR('String', start-index, length_of_extracted_string)

Input1: SELECT SUBSTR('Database Management System', 9) FROM DUAL;

Output1: Management System

Input2: SELECT SUBSTR('Database Management System', 9, 7) FROM DUAL;

Output2: Manage

Print 6 characters from 4th character from string	Print 2 characters from last to 5th character from string																																																
SQL> SELECT SUBSTR('RAJASHEKHAR',4,6) FROM DUAL;	SQL> SELECT SUBSTR('RAJASHEKHAR',-5,2) FROM DUAL;																																																
SUBSTR	SU																																																
ashekh	ek																																																
Print first 3 characters from emp table	Print last 3 characters from emp table																																																
SQL> SELECT SUBSTR(ENAME,1,3) FROM EMP;	SQL> SELECT SUBSTR(ENAME,-3,3) FROM EMP;																																																
SUB	SUB																																																
---	---																																																
SMI	ITH																																																
ALL	LEN																																																
WAR	ARD																																																
JON	NES																																																
MAR	TIN																																																
BLA	AKE																																																
CLA	ARK																																																
SCO	OTT																																																
Display the emp details who’s name start with ‘S’	Display the emp details who’s name endt with ‘N’																																																
SQL> SELECT * FROM EMP WHERE SUBSTR(ENAME,1,1)='S';	sQL> SELECT * FROM EMP WHERE SUBSTR(ENAME,-1,1)='N';																																																
<table><tr><th>EMPNO</th><th>ENAME</th><th>JOB</th><th>MGR</th><th>HIREDATE</th><th>SAL</th><th>COMM</th><th>DEPTNO</th></tr><tr><td>7369</td><td>SMITH</td><td>CLERK</td><td>7902</td><td>17-DEC-80</td><td>800</td><td>20</td><td></td></tr><tr><td>7788</td><td>SCOTT</td><td>ANALYST</td><td>7566</td><td>19-APR-87</td><td>3000</td><td>20</td><td></td></tr></table>	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	7369	SMITH	CLERK	7902	17-DEC-80	800	20		7788	SCOTT	ANALYST	7566	19-APR-87	3000	20		<table><tr><th>EMPNO</th><th>ENAME</th><th>JOB</th><th>MGR</th><th>HIREDATE</th><th>SAL</th><th>COMM</th><th>DEPTNO</th></tr><tr><td>7499</td><td>ALLEN</td><td>SALESMAN</td><td>7698</td><td>20-FEB-81</td><td>1600</td><td>300</td><td>30</td></tr><tr><td>7654</td><td>MARTIN</td><td>SALESMAN</td><td>7698</td><td>28-SEP-81</td><td>1250</td><td>1400</td><td>30</td></tr></table>	EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO	7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30	7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO																																										
7369	SMITH	CLERK	7902	17-DEC-80	800	20																																											
7788	SCOTT	ANALYST	7566	19-APR-87	3000	20																																											
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO																																										
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30																																										
7654	MARTIN	SALESMAN	7698	28-SEP-81	1250	1400	30																																										

INSTR : By using the in string function we can get the position of the character,

Or Returns the location in a string of the specified substring.

Syntax: INSTR (Column|Expression, 'search value', 'start position', 'end position')

Input: SELECT INSTR ('Google apps are great applications','app',1,2) FROM DUAL;

Output: 23

Input: SELECT INSTR ('rajashekhar','a',1,3) FROM DUAL;

Output: 10

Input: SELECT INSTR ('rajashekhar','a',-1,2) FROM DUAL;

Output: 4

rajashekhar@gmail.com

rajashekhar

@gmail

.com

user name

service provider name

domain name

SQL> select substr('rajashekhar@gmail.com',instr('rajashekhar@gmail.com','.')) from dual;

Output SUBS

:_____
.com

SQL> select substr('rajashekhar@gmail.com',instr('rajashekhar@gmail.com','.')+1) from dual;

Output SUB

com

SQL> select

substr('rajashekhar@gmail.com',instr('rajashekhar@gmail.com','@')+1,instr('rajashekhar@gmail.com','.')-instr('rajashekhar@gmail.com','@')-1) from dual;

Output SUBST

Gmail

SQL> select substr('rajashekhar@gmail.com',1,instr('rajashekhar@gmail.com','@',1,1)-1) from dual;

Output SUBSTR('RAJ

rajashekhar

TRANSLATE : By using translate function we can translate one character into another character.

Syntax: TRANSLATE ('input String', characters, translations)

Example:

```
SELECT TRANSLATE('abcdef','abc','bcd') AS Translated,
REPLACE(REPLACE(REPLACE('abcdef','a','b'),'b','c'),'c','d') AS Replaced;
```

Translated	Replaced
It is used to translate the character of string by new	It is used to replace the part of a string by new string
bcddef	Ddddef
Character is to be replaced	String is to be replaced
Ex: Raj - j to m ram	Ex: Raj to Sam

Example

The following query replaces characters A with B, B with C, C with D, D with E,...Z with A, and a with b, b with c, c with d, d with e z with a.

select

```
translate('interface','ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz',
'BCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz') "Encrypt" from dual;
```

```
Output    Encrypt
-----
joufsgbdf
```

CONCAT : Concatenates two strings into one.

Syntax:

CONCAT('String1', 'String2')

Input1: SELECT CONCAT('computer', 'science') FROM DUAL;

Output1: computerscience

Input2: SELECT CONCAT(NULL, 'Android') FROM DUAL;

Output2: Android

Input3: SELECT CONCAT(NULL, NULL) FROM DUAL;

Output3: -

LENGTH : Returns the length of a string.

Syntax:

LENGTH(Column|Expression)

Input1: SELECT LENGTH('Learning Is Fun') FROM DUAL;

Output1: 15

Input2: SELECT LENGTH(' Write an Interview Experience ') FROM DUAL;

Output2: 34

Input3: SELECT LENGTH('') FROM DUAL; or SELECT LENGTH(NULL) FROM DUAL;

DATE:

DATE is the datatype that we are all familiar with when we think about representing date and time values. It has the ability to store the month, day, year, century, hours, minutes, and seconds.

Example

```
SELECT SYSDATE FROM DUAL;
```

Working with Dates

- Oracle stores dates in an internal numeric format representing the century, year, month, day, hours, minutes, seconds.
- The default date format is DD-MON-YY.
- SYSDATE is a function returning date and time.
- DUAL is a dummy table used to view SYSDATE

In order to represent the date stored in a more readable format, the TO_CHAR function has traditionally been wrapped around the date:

```
SQL> SELECT TO_CHAR(SYSDATE,'DD-MON-YYY') FROM DUAL;
```

```
Example      TO_CHAR(SY
               -----
               16-NOV-021
```

```
SQL> SELECT TO_CHAR(SYSDATE,'DD-MONTH-YYYY') FROM DUAL;
```

```
Example      TO_CHAR(SYSDATE,'
               -----
               16-NOVEMBER -2021
```

```
SELECT TO_CHAR(hiredate,'DD.MM.YYYY') from emp;
```

```
Example      TO_CHAR(HI
               -----
               17.12.1980
               20.02.1981
               22.02.1981
```

```
SELECT TO_CHAR(hiredate,'DD.MM.YYYY:HH24:MI:SS') "hiredate" FROM emp;
```

```
Example      hiredate
               -----
               17.12.1980:00:00:00
               20.02.1981:00:00:00
               22.02.1981:00:00:00
               02.04.1981:00:00:00
               28.09.1981:00:00:00
```

SELECT ENAME, (SYSDATE - HIREDATE) FROM EMP;

ENAME	(SYSDATE-HIREDATE)
SMITH	14944.2132
ALLEN	14879.2132
WARD	14877.2132
JONES	14838.2132

- Display the all the employee from hired on 1981

SQL> SELECT * FROM EMP WHERE TO_CHAR(HIREDATE,'YYYY')=1981;

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	SALESMAN	7698	20-FEB-81	1600	300	30
7521	WARD	SALESMAN	7698	22-FEB-81	1250	500	30
7566	JONES	MANAGER	7839	02-APR-81	2975		20

MONTHS_BETWEEN

The MONTHS_BETWEEN function calculates the number of months between two dates and returns that difference as a number. The specification is:

MONTHS_BETWEEN (date1, date2)

- Calculate two ends of month, the first earlier than the second:
MONTHS_BETWEEN ('31-JAN-1994', '28-FEB-1994') ==> **-1**
- Calculate two ends of month, the first later than the second:
MONTHS_BETWEEN ('31-MAR-1995', '28-FEB-1994') ==> **13**
- Display the months between the hire date and system date

SQL> SELECT MONTHS_BETWEEN(SYSDATE,HIREDATE)/12 FROM EMP;

MONTHS_BETWEEN(SYSDATE,HIREDATE)/12
40.9145706
40.7398394
40.7344631

ROUND

ROUND function rounds a date value to the nearest date as specified by a format mask.

SQL> SELECT ROUND(months_between(sysdate,hiredate)/12) as experience from emp;
EXPERIENCE

41
41
41
41
40

DECODE:

We can decode the value from one format to another format

Syntax:

DECODE('CN', 'SEARCH VALUE', 'DESIRED FORMAT VLAUE');

Example: SELECT DECODE(1, 1, 'One') FROM dual;

Output: One

DECODE function has the functionality of an IF-THEN-ELSE statement.

```
SELECT supplier_name,  
DECODE(supplier_id, 10000, 'IBM',  
          10001, 'Microsoft',  
          10002, 'Hewlett Packard',  
          'Gateway') result  
FROM suppliers;
```

The above DECODE statement is equivalent to the following IF-THEN-ELSE statement:

```
IF supplier_id = 10000 THEN  
    result := 'IBM';  
ELSIF supplier_id = 10001 THEN  
    result := 'Microsoft';  
ELSIF supplier_id = 10002 THEN  
    result := 'Hewlett Packard';  
ELSE  
    result := 'Gateway';  
  
END IF;
```

```
SELECT college_id,  
DECODE (college_id, 10003,'Massachusetts, USA',
```

```
10004, 'California, USA',  
'India')  
FROM college_details;
```

Output:

CASE statement:

By using the case statement we can run or update multiple expressions using multiple conditions

Syntax

```
CASE [ expression ]
  WHEN condition_1 THEN result_1
  WHEN condition_2 THEN result_2
  ...
  WHEN condition_n THEN result_n
  ELSE result
END
```

Example:

```
SELECT
  CASE
    when sal>4000 then 'Grade A'
    when sal>2000 and sal<4000 then 'Grade B'
    when sal>1000 and sal<2000 then 'Grade C'
    else
    'Grade D'
  end as Grade
from emp;
```

GRADE

Grade D

Grade C

Grade C

Grade B

Grade C

Grade B

Grade B

Grade B

Grade A

Grade C

Grade C

DECODE	CASE
It is an Oracle property	It is an a ANSI SQL
We use equal operator	We use any condition operator
Used with any select statement	Used with select & update commands

SET OPERATORS

The SQL Set operators is used to combine the two or more SQL SELECT statements.

Types of Set Operation. .

1. Union
2. Union All
3. Intersect
4. Minus

1. Union:

By using union set operator we can combine the result sets of two or multiple select queries, by this we can eliminate duplicate and sort the result in ascending order

Table_A

X
1
2
3
4
5
6
7

Table_B

Y
1
2
3
4
5
8
9

Example:

```
SELECT X FROM TABLE_A UNION SELECT Y FROM TABLE_B;
```

2. Union All:

By using union all set operator we can combine the result set of two or multiple select queries. Here there is no elimination of duplicate and sorting result i.e., the first result set below which second result set appended.

```
SELECT X FROM TABLE_A UNIONALL SELECT Y FROM TABLE_B UNIONALL
SELECT Z FROM TABLE_C;
```

3. Intersect:

By using intersect operator we can retrieve the common values or rows from two or multiple select queries.

Example:

```
SELECT X FROM TABLE_A INTERSECT SELECT Y FROM TABLE_B;
```

Table_A	Table_B	Output
X	Y	
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	8	
7	9	

Interview Question: how to get common or matching value?

By using intersect set operators we can retrieves common or matching values.

4. Minus set operator:

By using minus set operator we can retrieves all the value from first select query but those are not present in second select query.

Example:

```
SELECT X FROM TABLE_A MINUS SELECT Y FROM TABLE_B;
```

Table_A	Table_B	Output
X	Y	
1	1	6
2	2	7
3	3	8
4	4	9
5	5	
6	8	
7	9	

If in any query output is '0' (zero) rows it means expected is loaded into target.

Condition or rules for set operators:

- The number of columns in first select query is equal to number of columns in second select query.
- The column name maybe or may not be same but data type must be same.
- Order of the column must be same.

Analytical Function

- Analytical functions are used to assign the rank to rows of the table.
- Analytical functions are always used with OVER clause, over(Partition by order by CN) partition is optional and order by is compulsory

Example: SELECT SCOTT.EMP.*, RANK () OVER (ORDER BY ASC) AS RANK FROM
 SCOTT.EMP;

1. **RANK**
2. **DENSE_RANK**
3. **ROW_NUMBER**
4. **LAG**
5. **LEAD**

1. RANK: It assign the rank to the table, In this it skips the next rank whenever duplicate occurs

Example: SELECT SCOTT.EMP.*, RANK() OVER (ORDER BY SAL DESC) FROM SCOTT.EMP;

2. DENSE_RANK: It assign the rank to the table, In this which doesn't skip the next rank whenever duplicate occurs.

Example: SELECT SCOTT.EMP.*, DENSE_RANK () OVER (ORDER BY SAL DESC) DENSE_RK
 FROM SCOTT.EMP;

3. ROW_NUMBER: It assign the rank to the table. It assigns rank in sequential manner.

EXAMPLE: SELECT SCOTT.EMP.*, ROW_NUMBER () OVER (ORDER BY SAL DESC) RW_N
FROM SCOTT.EMP;

 SELECT SCOTT.EMP.*, ROW_NUMBER () OVER (PARTITION BY DEPTNO ORDER BY
 SAL DESC) FROM SCOTT.EMP;

4. LEAD:

- By using lead function, we can retrieve the next value of a row in a column of the same table. (not using join).
- The last row value is null by default.

Example: SELECT (COLUMN_NAME, CONTEXT, DEFAULT VALUE) OVER (ORDER BY CN)
 FROM TABLE_NAME;

Example:

```
SELECT LEAD (SAL, 1) OVER (ORDER BY SAL DESC) FROM SCOTT.EMP;
```

```
SELECT SCOTT.EMP.*, LEAD (SAL, 1) OVER (ORDER BY SAL DESC) FROM SCOTT.EMP;
```

```
SELECT SCOTT.EMP*, LEAD (SAL,1,LATEST) OVER (ORDER BY SAL DESC) FROM  
SCOTT.EMP;
```

LAG:

- By using lag function we can retrieve previous row value in a column of the same table (without join)
- Here first row value is null.

Example:

```
SELECT SCOTT.EMP*, LAG (SAL, 1) OVER (ORDER BY DESC) FROM SCOTT.EMP;
```

Example:

Table A	Ascending(Asc)	Rank	Dense_Rank	Row_Number
100	100	1	1	1
200	100	1	1	2
100	100	1	1	3
300	200	4	2	4
100	200	4	2	5
400	300	6	3	6
200	400	7	4	7
500	500	8	5	8
500	500	8	5	9
500	500	8	5	10
600	600	11	6	11
700	600	11	6	12
800	700	13	7	13
600	700	13	7	14
700	800	15	8	15

Join

By using the join function we can combine the row from two or multiple table based on common data type in attribute (column).

Types of joins:

- a) Inner join or Equi join or Simple Join
- b) Non-equi Join
- c) Cross product or Cartesian join
- d) Left outer join
- e) Right outer join
- f) Full outer join
- g) Self join

a) Inner join or Equi join or Simple Join:

- By using inner join function we can retrieve the matching rows from two or multiple tables based on common data type columns and used equal to (=) operator in a condition.

Example: SELECT A.X, B.X FROM TABLE_A A, TABLE_B B WHERE A.X=B.X;

Example: SELECT A.X, B.X FROM TABLE_A A INNER JOIN TABLE_B ON A.X=B.X;

Table_A	Table_B	Output	
X	X	x	x
1	3	3	3
2	2	2	2
3	1	1	1
6	7		

b) Non Equi Join:

- By using non-equi join function we can retrieve the non matching rows from two or multiple tables based on common data type columns and used equal to (!= or < >) operator in a condition.
- Here we retrieve the non – matching records.

Example: SELECT A.X, B.X FROM TABLE_A A, TABLE_B B WHERE A.X!=B.X;

c) Cross product join or Cartesian join:

- By using cross join we can combine rows of two or multiple table without any condition.
- Without condition join the table.
- This is used where possibilities require.

Example: SELECT A.X, A.X FROM TABLE_A A, TABLE_B B;

d) Left Outer Join:

- By using left outer join we can retrieve all the values from the left table those are matched or unmatched but unmatched values are displayed as null in right table.

Example: SELECT A.X, B.X FROM TABLE_A A LEFT OUTER JOIN TABLE_B B ON A.X=B.X;

Table_A	Table_B	Output	
X	Y	x	y
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	7	5	(null)
6	8	6	(null)

e) Right outer Join:

- By using right outer join we can retrieve all the values from the right table those are matched or unmatched but unmatched values are displayed as null in left table.

Example: SELECT A.X, B.X FROM TABLE_A A RIGHT OUTER JOIN TABLE_B B ON A.X=B.X;

Table_A	Table_B	Output	
X	Y	x	y
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	7	(null)	7
6	8	(null)	8

f) Full Outer Join:

- By using full outer join we can combine all the rows (or records) from left and right table whether it match or un-match but un-match value display as null in corresponding table.

Example: SELECT A.X, B.X FROM TABLE_A A FULL OUTER JOIN TABLE_B B ON A.X=B.X;

Table_A	Table_B	Output	
X	Y	x	y
1	1	1	1
2	2	2	2
3	3	3	3
4	4	4	4
5	7	5	(null)
6	8	6	(null)
		(null)	
		(null)	

g) Self join:

- By using self join we can compare the two columns of same table.

Example: SELECT E.EMP, E.JOB FROM SCOTT.EMP E, SCOTT.EMP M WHERE E.MGR = M.EMPNO;

Agile Methodology

Agile Methodology is the method used for the developing the product or application of the software.

There are 6 phases or stages or tabs.

- 1) Project Initiation
- 2) Sprint Planning
- 3) Daily Scrum
- 4) Sprint Retrospective
- 5) Sprint Demo
- 6) Release

1) Project Initiation:

- Product Owner fall in this category. (Marketing Person or Business Analyst)
- He interacts with client or stake holder or Customer.
- Collect the requirement from the client.
- He will start the project work.
- And he has a vision and knows about the project what should be delivered.

2) Sprint Planning: (starting & Ending time) (Sprint means segment)

- It is fixed time duration for project working is called sprint. It is usually 1 to 4 weeks.
- In our project the sprint is 2 week (10 Days).
- Here product owner and team decide the sprint.
- Before each sprint the team conducting the test plan.
- And what should be delivered to during sprint.
- In that answering for WH Questions like What, Who, When, How to process.
- The sprint plan depends on task level.
- It changes according to the condition.

3) Daily Scrum: (Involve all team)

- Each day the team has short meeting in which we update ourselves about project progress.
- It is always 15 minutes or little bit more.
- And discuss about
 - What we did yesterday?
 - What you will do today?
 - And is there any obstacles.

Scrum Master:

- He is a supervisor and he removes obstacles or he resolves the problem faces.
- Without scrum master the project is high risk of failures.
- Scrum Master present in offshore.
- Manager is not a scrum master.
- The scrum master manages the sprint and product burn-down chart.

4) Sprint Retrospective:

- At the end of the every sprint.
- After every release of the sprint team reviews about the
 - What went well during sprint?
 - What went wrong during the sprint?
 - How we can improve the sprint in future?

5) Sprint Demo:

- It is the presentation layer from scrum.
- Scrum (Team) do the presentation to the Client or Stake Holder.
- In that testing team focuses on acceptance criteria.

6) Release:

- Release of the product or application to the client.

Environments in the project completion

a) Dev Environment	-	Developer involved
b) Test Environment	-	Testing team involved
c) Client Environment	-	Client and testing and development team involved
d) Production Environment	-	End User are involved

Agile Method Terminologies.

Epic:

- Epic are large piece of work. (Features, customer requirement, business requirement)
- Epic breakdown into smaller called Stories.
- Each story contains individual requirements these are called Product Back-log Item (PBI).

PBI: Product Back-Log Item

- It is a list of requirements required by the customer.
- It is single element of work that exist in PB (Product Back-lock)

SBI: Sprint Back-Log Item

- It is a segment of PBI that are select by the team during the scrum sprint. (Number of PBI considered in the current sprint)

Burn-Down Chart:

- It is a graphical representation of chart it shows the outstanding work against project time.
- This will be helpful for completing the time for completion of product.
- From that we can see the progress of the project.

User Story:

- It is a non technical statement of software system requirements written from the end user point of view.

Story Points:

- Story points are used to determine workload effort for a user story.

Agile Method Advantages and Disadvantages**Advantages:**

- It is highly flexible
- Fast implementation of any changes
- Incremental updates of software
- Faster time to market.
- More rapid development.
- Higher satisfaction from customer.
- Higher Productivity.
- Low Project Cost.

Disadvantages:

- Lack of detailed documentation may lead communication gap.
- Add-On Training requires in some cases.
- User is required to test and analyse on daily bases.

LEVELS OF TESTING

1. Unit Testing
2. Smoke Testing
3. Sanity Testing
4. Functional Testing
5. Integration Testing
6. System Testing
7. Re-Testing
8. Regression Testing
9. UAT Testing

Types of Testing:

- a) **White Box Testing:** It needs the internal knowledge of the language. It is done by developer.
- b) **Black Box Testing:** In which we don't require to know internal knowledge of language.
- c) **Grey Box Testing:** In this we know the semi-internal

Note: Exploratory Testing: Learning and testing are parallel doing. This testing will be takes place when domain changes.

LEVELS OF TESTING

TESTING	TYPE OF TESTING	WHO WILL	ENVIRONMENT	PURPOSE
1. Unit Testing	White Box Testing	Developer	Developer Environment	Make sure that coding part is working properly or not
2. Smoke Testing	Black Box Testing	Developer or Deployment Team	Deployment Environment	Make sure that all main features working properly or not
3. Sanity Testing	Black Box Testing	Tester	Testing Environment	Make sure that all main features working properly or not
4. Functional Testing	Black Box Testing	Tester	Testing Environment	Make sure that each unit function working properly as per requirement or not
5. Integration Testing	Black Box Testing	Tester	Testing Environment	Make sure that after combining two components communicating each other properly or not
6. System Testing	Black Box Testing	Tester	Testing Environment	Make sure that whole component communicating each other properly or not, it is nothing but end to end testing
7. Re – Testing	Black Box Testing	Tester	Testing Environment	After fixing the bug the test is carried is called retest & make sure that working as per expected
8. Regression Testing	Black Box Testing	Tester	Testing Environment	If anything changes to the application then make sure that it working properly or not. We know that also how it will impact on other module
9. UAT Testing (user acceptance test)	Black Box Testing	Tester in front of client	Client environment	Make sure that main features are working properly or not and also carry criteria (i.e., Agreements and Documents)

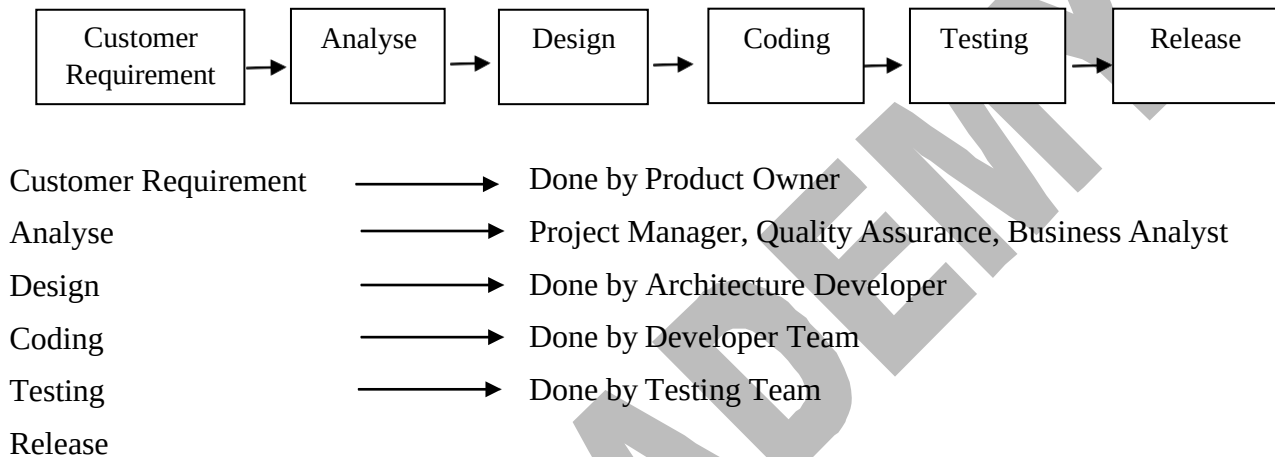
Manual Testing

1. SDLC (Software Development Life Cycle)
2. STLC (Software Test Life Cycle)

1. SDLC (Software Development Life Cycle):

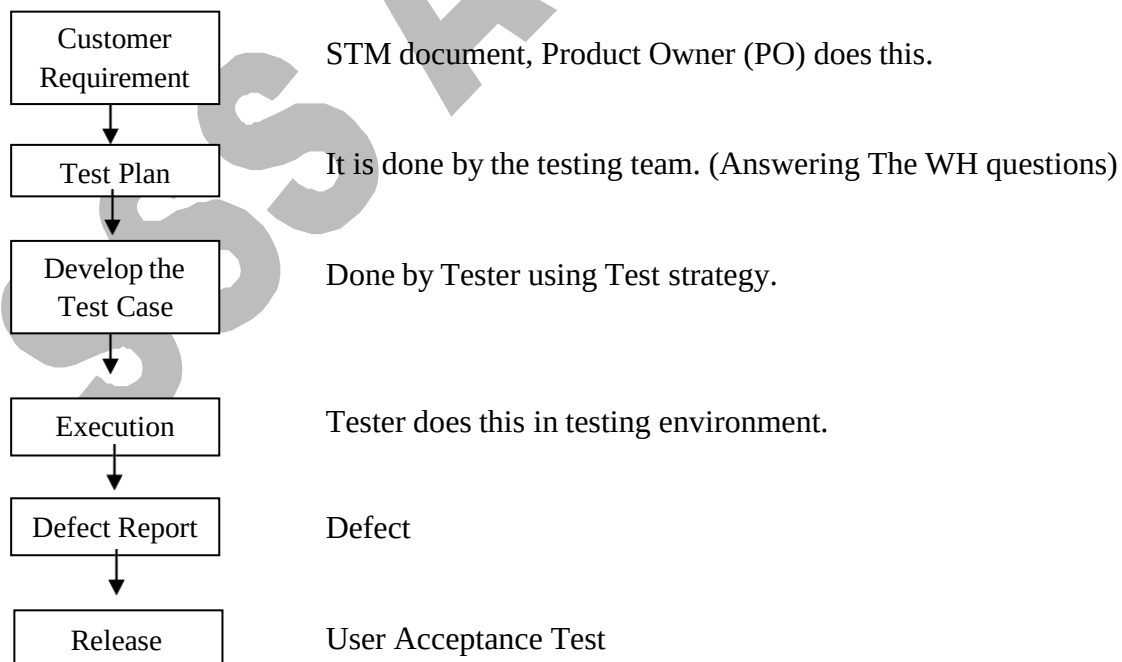
This is the method of developing the software or product.

Stages of SDLC



2. STLC (Software Test Development Cycle):

This is the process or method of testing the software or product.



Test Case: It is nothing but Sequential, Elaborate and Executable form of requirement.

Case Template:

Test Scenario	Test Case Number	Description	SQL Query	Expected Result	Actual Result	Status	Remarks
---------------	------------------	-------------	-----------	-----------------	---------------	--------	---------

Test Case Strategies: How you write the test cases)

a) BVA (Boundary Value Analysis)

- It defines the range of value to the test.
- It is used whenever test having range value.

Example:

Entering the password with range of 4 to 12 characters.

Password	i.e. Password Range is min 4 to 12 max,
---	---

Min = 4	Pass	Max=12	Pass
Min -1 = 3	Fail	Max – 1 = 11	Pass
Min+1 = 5	Pass	Max + 1 = 13	Fail

b) Equivalence Class Partition:

- It defines the type of data
- These are the data used in performing the testing on the application.
- Tester will go to create in excel sheet.

Password	Accept the Alpha Numeric Condition
---	------------------------------------

{a – z} {A – Z} {0 – 9}	Valid
{a – z} {0 – 9}	Valid
{a – z} {A – Z}	Invalid

Positive Testing:

Whenever we use valid Test data for testing, we call it as Positive Testing.

Negative Testing:

Whenever we use Invalid Test data for testing, we call it as Negative Testing.

Error Guessing:

When in test case we have

Name	-	Mandatory
Address	-	Mandatory
Pin code	-	Mandatory
City	-	Optional
Vehicle	-	Optional

- Name, Address, Pin Code - Ok Enable → If criteria ok then it will enable
- Name, Address, Pin Code, City - Ok Enable
- Name, Address, City, Vehicle - Disabled → Because Mandatory Fields not present

Severity:

- Seriousness about the defect or of the defect with respect to the other functionality.
- It is decided by the tester.

Example:

Urgent	Without steering
Very High	Break not working
High	Head Light not working
Medium	A/c not working
Low	Sound or Audio not working

Priority:

- How soon the defect will resolve is called priority.
- It is decided by the Developer.

Note: Severity and priority are decided in call with onsite coordinator, client, developer and tester.

Requirement Traceability Matrix: (RTM – Requirement traceability Matrix, it is a document.)

It traces or maps the requirement of the customer through Test Case. OR it makes ensure that all the requirements from customer are tested through test cases.

To Validate the entire requirement or checked:

Sl no	Scenario	Test Case	Status
01	Password	Source Test Case – 1	Checked
		Source Test Case – 2	Checked
		Source Test Case – 3	Checked
02	User Name	Source Test Case – 1	Checked
		Source Test Case – 2	Checked
		Source Test Case – 3	Checked
03	Radio Button		Not Checked
			Not Checked
			Not Checked
04	Drop Down	Source Test Case – 1	Checked
		Source Test Case – 2	Checked
		Source Test Case – 3	Checked

- Here we make ensure that all requirements are validate the test. It indicates the coverage of test case.

Exploratory Testing:

- This is a Tester Carried test with parallel learning the new domain.
- This is used when we change domain.

Random or Adhoc Testing:

- This is a test carried randomly by experienced person for checking the major features.

Cast:

A cast function is used to convert one data-type into another data-type temporarily.

Varchar2	-	Char	Done
Char	-	Varchar2	Done
Varchar2	-	Number	Cannot be done
Number	-	Varchar2	Done

SYNTAX:

Constraints

These are rules or conditions or limits to the data flow into the table.

1. Not Null
2. Unique
3. Primary key
4. Foreign key
5. Check

Roll No	Name	Branch

1. Not Null:

- Must assign value
- Accept the duplicate
- 'N' number of duplicates
- Doesn't accept NULL

Syntax:

```
CREATE TABLE <TABLE_NAME> (CN DATATYPE (SIZE), CN DATATYPE (SIZE) NOT NULL, CN DATATYPE (SIZE));
```

Example:

```
CREATE TABLE CONSTRAINTS (ROLLNO NUMBER (10), NAME VARCHAR2 (10) NOT NULL, BRANCH VARCHAR2, (08));
```

2. Unique:

- It maintains the unique record in the database or table.
- Not accept the duplicate
- Accept null

Syntax:

```
CREATE TABLE <TABLE_NAME> (CN DATATYPE (SIZE) UNIQUE, CN DATATYPE (SIZE), CN DATATYPE (SIZE));
```

Example:

```
CREATE TABLE CONSTRAINTS (ROLLNO NUMBER (10) UNIQUE, NAME VARCHAR2 (10), BRANCH VARCHAR2,(08));
```

3. Primary key:

- It is the combination of not null and unique.
- It doesn't accept null
- It doesn't accept duplicate
- For one table one primary key.
- Primary key is always present in parent table.

Syntax: `CREATE TABLE <TABLE_NAME> (CN DATATYPE (SIZE) PRIMARY KEY, CN DATATYPE (SIZE), CN DATATYPE (SIZE));`

Example:

```
CREATE TABLE CONSTRAINTS (ROLLNO NUMBER (10) PRIMARY KEY, NAME VARCHAR2 (10), BRANCH VARCHAR2, (08));
```

Note: Composite Primary Key: In a table more than one column having the primary key then we call this as composite primary key

4. Foreign key:

- It references the Primary Key.

Example:

```
CREATE TABLE DEPTNO (DEPTNO NUMBER (10) REFERENCES SCOTT.EMP (DEPTNO));
```

5. Check Constraint:

- By using this we can define range value and conditions

Example:

```
CREATE TABLE EMP (SAL NUMBER (10) CHECK (SAL BETWEEN 5000 AND 50000));
```

```
CREATE TABLE EMP (SAL NUMBER (10) CHECK SAL>5000);
```

Like Operators:

- By using this we can set the data based on pattern matching.

Here we used two Wild Cards

% (percentage) Extract many Characters.

_ (under score) Extract single Characters.

Example:

```
SELECT ENAME FROM SCOTT.EMP WHERE ENAME LIKE 'S%';
```

```
SELECT ENAME FROM SCOTT.EMP WHERE ENAME LIKE %S%';
```

```
SELECT ENAME FROM SCOTT.EMP WHERE ENAME LIKE ' _L%';
```

```
SELECT SAL FROM SCOTT.EMP WHERE SAL LIKE '%3';
```

SCD Type – 2 Validations

Test Case Template: Example (how to write test case)

Scenario: Record count between source and target.

Test Case no.	Description	SQL Query	Expected Result	Actual Result	Status	Remark
1.1	Record count for the source table Source_table name	Select count(ID) from S_TN	Query should run successfully and retrieve the number of records in source	Query run successfully and record of (count of ID)	Pass	
1.2	Record count for the target table (TN) Eg: Server_Schema_TN (path)	Select count(ID) from T_TN	Query should run successfully and retrieve the number of records in target	Query run successfully	Pass	
1.3	Compare the record count for 1.1 and 1.2	Not needed	Both should match		Pass	

Loads:

Files which are transferred to the data warehouse.

- **Initial Load or Full Load:** The data or files which are loaded for the first time into the data warehouse.
- **Incremental Load or Delta Load:** The data or files which are loaded for the second time into the data warehouse.

Source Table:

ID	Name	City
100	Rajesh	New-York
200	Nikhil	New-Jersey
300	Mark	Washington
400	Allen	California

Question, What are test cases in SCD Type-2 or How do you validate SCD Type – 2

Target Table

Test Case for Initial Load:

Surrogate Key	ID	Name	City	ETL_Effective_Start_Date	ETL_Effective_End_Date	Active Row Flag	Version no.	ETL Process Date
1000	100	Rajesh	New-York	01-Jan-20	31-Dec-2099	A	1	01-Jan-20
1001	200	Nikhil	New-Jersey	03-Feb-20	31-Dec-2099	A	1	03-Feb-20
1002	300	Mark	Washington	04-Feb-20	31-Dec-2099	A	1	04-Feb-20
1003	400	Allen	California	06-Mar-20	31-Dec-2099	A	1	06-Mar-20

We are going to validate on the basis of generic automatic fields produced by ETL such as Surrogate Key ETL Effective Start Date, ETL Effective End Date, Active Row Flag, Version number and ETL Process Date.

- i. Job run successfully
- ii. Validate Surrogate key must generate Automatically, Sequence and Incremental Order. It should be numeric.
- iii. Make sure ETL Effective Start Date and ETL Process Date both must be same.
- iv. ETL effective end date should be higher than that of ETL effective start date, usually that date is mentioned in STM if not given will take default 31-Dec-99
- v. Make sure that active row flag field contains 'A' for all records.
- vi. Make sure all the values in version must be '1'.
- vii. ETL process date should be similar as ETL effective start date.
- viii. Compare the record count between source & target.
- ix. Verify the duplicate records in target table.
- x. Make sure that verify the null values populated in target.
- xi. Perform the mapping column, i.e., perform minus query with expecting zero return.
- xii. Apart from all the above validation we are going to carry business logic validation.

Test Case for Incremental Load:

Surrogate Key	ID	Name	City	ETL Effective_ Start Date	ETL Effective End Date	Active Row Flag	Version no.	ETL Process Date
1000	100	Rajesh	New-York	01-Jan-20	01-Oct-2020	H	1	01-Jan-20
1001	200	Nikhil	New-Jersey	03-Feb-20	31-Dec-2099	A	1	03-Feb-20
1002	300	Mark	Washington	04-Feb-20	31-Dec-2099	A	1	04-Feb-20
1003	400	Allen	California	06-Mar-20	31-Dec-2099	A	1	06-Mar-20
1004	100	Rajesh	Florida	02-Oct-20	24-Apr-2021	H	2	02-Oct-20
1005	100	Rajesh	Texas	25-Apr-21	31-Dec-2099	A	3	25-Apr-21

We are going to validate on the basis of generic automatic fields produced by ETL such as Surrogate Key ETL Effective Start Date, ETL Effective End Date, Active Row Flag, Version number and ETL Process Date.

- i. Job run successfully
- ii. Validate Surrogate key must generate Automatically, Sequence and Incremental Order. It should be numeric.
- iii. Make sure ETL Effective Start Date and ETL Process Date both must be same.
- iv. *) ETL effective end date for a newly insert record is higher than that of ETL process date as per STM or Default.
*) ETL effective end date for updated record is ETL process date minus (-) 1 day.
- v. *) Make sure that active row flag field contains 'A' for newly inserted record.
*) For updated record the row should be 'H'.
- vi. *) For newly insert record the version number should be maximum and incremental.
*) For updated record it should be same as that of previous version.
- vii. ETL process date should be similar as ETL effective start date.
- viii. Compare the record count between source & target.
- ix. Verify the duplicate records in target table.
- x. Make sure that verify the null values populated in target.
- xi. Perform the mapping column, i.e., perform minus query with expecting zero return.
- xii. Apart from all the above validation we are going to carry business logic validation.

Difference between Primary Key and Surrogate Key;

Primary Key	Surrogate Key
It is used for maintaining the unique record in OLTP data base	It is used to maintain records in OLAP data base or data warehouse
Primary Key is always alphanumeric	Surrogate key is always numeric number
Primary Key is a business attribute	S.K is technical attribute
It is table data	It is not table data
It is given by user sometimes automatically generated	S.K is generated Automatically

Date Modelling: It is the process of designing data base with the set of tables. i.e.,

- Start Schema
- Snow Flake Schema

Testing: It is the process of developing the software on basis of business requirement.

SDLC Models (Software development Life Cycle

Sequential Model:

- In this model all SDLC activities will be carried out one after the another for the entire project.
- This model is recommended for small size project.

In this model **Incremental Model:**

- a big project or long term project will be divided into modules and SDLC activities will be carried out modules by modulus.

Difference between ETL Testing and manual Testing:

ETL Testing	Manual Resting
The process of extracting data from multiple sources (Flat file, XML, SAP) more simpler with help of tools	Loading the data other than flat files and oracle table need more effort
High & clear visibility of logic	Complex and not so user friendly visibility of logic
Contains Meta data and changes can be done easily	No Meta data concept and changes need more effort
Can handle historic data very well	Maximum effort from maintenance point of view

Importance of ETL testing:

- Helps in identifying problems with source information.
- Facilitates the transfer of bulk data.
- Prevents loss of data and duplicate data.

Large Volume Data:

- Base on business logic create a temporary table. In that we store data & validate
- Also generally by sampling. i.e., randomly picking.

Source data conversion:

Approach – I

- i. Create a temporary table in target data base.
- ii. Import flat file data into temporary table by following below step.
 - A. Select database menu
 - B. Select import sub menu and then click on import table data option.
 - C. Select the table and schema details from destination window.
Schema name, table type (.cst or table name, click on show data button.
- iii. Select the source file from select source window and also select file type and select on next button.
- iv. Select the delimiter option from next window.
- v. In next window enter value for first row=2 click on next button, click on execution button after that data will be loaded to temporary table.

We write test cases in excel and load to HPALM

Questions:

- 1.
2. **If you find Mismatch records. What is your approach**
 - Along with mismatch records and its artifacts, I will contact the on-site co-ordinator and team
 - Then we discuss on the same and raise a bug depending upon their call.
 - Repeat bug cycle.
3. **If you find a bug, what is your first approach.**
 - First I go get the log sheet or session.
 - From their I will collect error description and take artifacts.
 - Then with that evidence I will contact on site co-ordinator and team and discuss with them and raise a bug report with status as new.
 - Repeat the bug cycle.
4. **If tomorrow is release, if you find a bug then what is your approach.**
 - First I go to get log session.
 - From then I collect error description and take artifacts.
 - Then I will try to solve it on myself if not solved
 - Then with that evidence I will contact my onsite co-ordinator and team and asks them to take more time from client.
 - If not then I will convince to my on-site co-ordinator and team and inform them about severity of the defect.

5. What is ETL testing?

- ETL stands for Extraction transformation and load
- Extraction means to extracting data from sources or application.
- Transformation means to apply conversion rules on data so that it becomes suitable for analytical reporting.
- Loading means to move data into target system i.e., Data warehouse.

6. B.I (Business Intelligence)

- BI is the process for collecting raw business data and transforming into a meaning full vision.
- i.e., more useful to business.

7. What are Bugs in ETL

- Source Bug
- Load condition Bug
- Calculation Bug
- User Interface Bug

Meta Data:

Data about the data is called meta data.

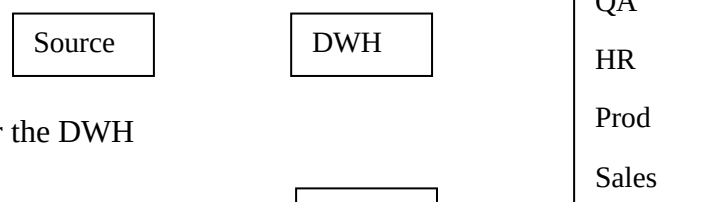
Data Mart:

- It is the subject of data warehouse
- It is specific subject oriented data which is designed for answering the specific questions from specific user.

Example: HR, Production Manager, Sales Manager, Quality Assured etc...

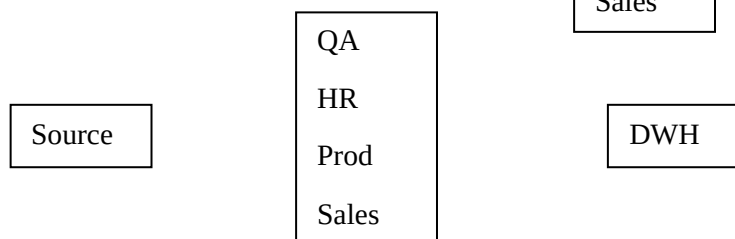
- It is used for faster access the data
- Every project there are two approaches of data mart

i. Top Down



Data Mart is placed after the DWH

ii. Bottom Down



Data Mart is placed before the DWH

Data Mart Types:

- Dependent Data Mart: (Depending on DWH) (Top Down)
- Independent Data Mart (Not Depending on DWH) (Bottom Down)

NORMAL DATABASE (OLTP)	DATA WAREHOUSE (OLAP)
OLTP (Online Transaction Processing)	OLAP (Online Analysis Process)
It records the user current transaction data	It maintain the historical data
Tables and Joins are complex since they are <u>normalized</u>. (Avoid duplicates)	Tables and Joins are simple since they are de-normalized.
Optimised for keeping small amount of data	Optimised for keeping large amount of data
MB to GB	GB to TB
Small to Large	Large to Very Large
Entity relationship Modelling technique is used	Dimension modelling technique is used
Optimized (used) for write operations like insert, update, Delete	Primarily Read only operation
Application oriented	Subject oriented
Only single stream	Integrity
Volatile	Non-volatile
Handling single record at a time	Handling multiple records at a time

DIMENSION TABLE	FACT TABLE
It contains details descriptive about the business	It contains measures about the business
It is having primary key	It is having foreign key
It contains textual data	It contains measures in numeric
- In OLTP dimension table are normalized - In OPAP dimension table are de-normalized	It is always de-normalized
Dimension table contains less data	Fact table contains more data
It contents Horizontal Table	It contents Vertical Table

STAR SCHEMA	SNOW FLAKE SCHEMA
A centralized located fact table which surrounded by multiple dimension table.	It is the extension star schema, in which dimension table exploded into another dimension table
It is top down model	It is bottom up model
Dimension table are de-normalized	Dimension table are normalized
Simple in structure hence joins and query are simple	Complex or complicated in structure hence joins and tables are complicated
Execution of the SQL query is high Performance	Performance of the query is less

CHAR	VARCHAR2
Fixed Length	Flexible Length
Name char(10)	Name varchar2(10)
Raju----- ('-' empty is unused)	Raju -----('-' send back to DB)
Maximum size of length is 0 - 255	Maximum size of length is 0 - 255

DROP	TRUNCATE
We delete the table in a single shot permanently from DB	We delete the table records in a single shot permanently from DB but structure remains same.
Performance is high	Performance is low

DELETE	DROP
It is a DML command	It is DDL command
Used to delete the records from the table	Used to delete structure of table
No Auto Commit	Auto Commit
Data can be roll back before commit	No roll back

IN	EXISTS
Low Performance	High Performance
Maintains column names	Does not Maintain column names
It is effective when most of filter criteria in sub query	It is effective when most of filter criteria in main query

SUB or NESTED or INNER QUERY	CORRELATE SUB QUERY
Inner and outer query are independent	Inner and outer query are interdependent
Outer query executed once	Outer query executed twice
High Performance	Low Performance

DELETE	TRUNCATE
Similarities: In these both commands the we delete data from the table but structure remains same.	
• We delete the specific row or whole data from the table.	• We delete all the data from the table at a single shot.
• By using where clause we delete specific row	• Truncate doesn't support the where clause.
• Performance is low	• Performance is high
• It is a DML command	• It is DDL command
• These are not auto commit	• These are auto commit
• Can be roll-back	• Cannot be roll-back

NVL	NVL2
Replace only null column	Replace both not null and null column
Ex: (comm, '5000')	Ex. (comm., '5000', '2000')
null	not null null
It passes two parameters	It passes three parameters

Print 6 characters from 4th character from string	Print 2 characters from last to 5th character from string
SQL> SELECT SUBSTR('RAJASHEKHAR',4,6) FROM DUAL; SUBSTR ashekh	SQL> SELECT SUBSTR('RAJASHEKHAR',- 5,2)FROM DUAL; SU ek
Print first 3 characters from emp table	Print last 3 characters from emp table
SQL> SELECT SUBSTR(ENAME,1,3) FROM EMP; SUB --- SMI ALL WAR JON	SQL> SELECT SUBSTR(ENAME,-3,3) FROM EMP; SUB --- ITH LEN ARD NES TIN
Display the emp details who's name start with 'S'	Display the emp details who's name endt with 'N'
SQL> SELECT * FROM EMP WHERE SUBSTR(ENAME,1,1)='S'; EMPNO ENAME JOB MGR HIREDATE SAL COMM DEPTNO 7369 SMITH CLERK 7902 17-DEC-80 800 20 7788 SCOTT ANALYST 7566 19-APR-87 3000 20	SQL> SELECT * FROM EMP WHERE SUBSTR(ENAME,-1,1)='N'; EMPNO ENAME JOB MGR HIREDATE SAL COMM DEPTNO 7499 ALLEN SALESMAN 7698 20-FEB-81 1600 300 30 7654 MARTIN SALESMAN 7698 28-SEP-81 1250 1400 30

TRANSLATE	REPLACE
It is used to translate the character of string by new	It is used to replace the part of a string by new string
bcddef	Ddddef
Character is to be replaced	String is to be replaced
Ex: Raj - j to m ram	Ex: Raj to Sam

DECODE	CASE
It is an Oracle property	It is an a ANSI SQL
We use equal operator	We use any condition operator
Used with any select statement	Used with select & update commands

WHERE	HAVING
It is used when we want to specify the condition for the column	It is used when we want to specify the condition for the group function
It can be used with select, update, delete	It cannot be used
Used before group by clause	It used after group by clause
We cannot use where clause in aggregate function	We can use having clause in aggregate function

UNION	UNIONALL
It retrieves the data without Duplicate	It retrieves the data with Duplicate
Low performance	High performance
Sorts results in ascending order	No any sorting of result

UNIONALL	EQUIJOIN
It retrieves total number of records from table irrespective of condition	It retrieves total number of records from table with respect to condition
It combines two rows of same table	Here we need two tables or two columns of two different tables
Here column data type should be same	Here column data type may or may not be same

JOIN	UNIOIN
It combines data from many tables based on matched condition between them	It combines result set of two or more select statement
Combines data into new column	It combines data into new rows
May not return distinct column	It return distinct rows

PRIMARY KEY (UNIQUE+NOT NULL)	UNIQUE KEY
Whenever we define primary key for column the system will generate index automatically	Whenever we define unique key for column then we have to define the index
Primary key is only for column in most of cases	Unique key is no of column
It will not accept null values	It will accept null values

PRIMARY KEY	FOREIGN KEY
Each table can have only one primary key	The table can have multiple foreign key
PK=unique + not null	Null & duplicate
Cluster index	Non cluster index
Relation between fact table and dimension table	In foreign key relation between some dimension & fact table

PRIMARY KEY	SURROGATE KEY
It is used for maintaining the unique record in OLTP data base	It is used to maintain records in OLAP data base or data warehouse
Primary Key is always alphanumeric	Surrogate key is always numeric number
Primary Key is a business attribute	S.K is technical attribute
It is table data	It is not table data
It is given by user sometimes automatically generated	S.K is generated Automatically

DISTINCT	UNIQUE
Its a clause	Its a constrain
Used whenever we are retrieving the data	Used whenever we are in sorting data
It will not accept null value	It will accept null value

VIEW	TABLE
View stores the record logically	Will store the record physically
Will not accept any space in DB	Table will occupy the space
Store SQL quarries	Table stores data

VIEW	MATERIALIZED VIEW
View stores the data logically	It stores the data physically
After the dropping the table view cannot be accessed	Even after dropping the MV is accessible
Purpose is for security	For performance purpose
Performance is low	Performance is more
We can update the record in view	We can't update the record

NOT NULL	UNIQUE
Not null will not accept the null values	Accept the null values
It accept the duplicate record	It will not accept the duplicate records

STORE PROCEDURE	FUNCTION
It may or may not return values	It must return a values
We can use all DML statement	It cannot use DML statement
It cannot be called from select statement	It can be called from select statement

ETL Testing	Manual Resting
The process of extracting data from multiple sources (Flat file, XML, SAP) more simpler with help of tools	Loading the data other than flat files and oracle table need more effort
High & clear visibility of logic	Complex and not so user friendly visibility of logic
Contains Meta data and changes can be done easily	No Meta data concept and changes need more effort
Can handle historic data very well	Maximum effort from maintenance point of view

Unix Commands

❖ **Man** command.

man This is help command, and will explain you about online manual pages you can also use man in conjunction with any command to learn more about that command for example.

- man ls will explain about the ls command and how you can use it.
- man -k pattern command will search for the pattern in given command.

❖ **Banner** command.

banner prints characters in a sort of ascii art poster, for example to print wait in big letters. I will type banner wait at unix command line or in my script. This is how it will look.

```
# # ## # #####  
# # # # #
```

❖ **Cal** command

cal command will print the calander on current month by default. If you want to print calander of august of 1965. That's eightht month of 1965. cal 8 1965 will print following results.

```
August 1965  
S M Tu W Th F S  
1 2 3 4 5 6 7  
8 9 10 11 12 13 14  
15 16 17 18 19 20 21  
22 23 24 25 26 27 28  
29 30 31
```

❖ **Clear** command

clear command clears the screen and puts cursor at beginning of first line.

❖ **Calendar** command

calendar command reads your calendar file and displays only lines with current day. For example in your calendar file if you have this

12/20 Test new software.

1/15 Test newly developed 3270 product.

1/20 Install memory on HP 9000 machine.

On dec 20th the first line will be displayed. you can use this command with your crontab file or in your login files.

❖ **Nohup** command.

nohup command if added in front of any command will continue running the command or process even if you shut down your terminal or close your session to machine. For exmaple, if I want to run a job that takes lot of time and must be run from terminal and is called update_entries_tonight . nohup update_entries_tonight will run the job even if terminal is shut down in middle of this job.

❖ **Tty** command

Tty command will display your terminal. Syntax is *tty options*

Options

- -l will print the synchronous line number.
- -s will return only the codes: 0 (a terminal), 1 (not a terminal), 2 (invalid options) (good for scripts)

File Management commands.

❖ **Pwd** command.

pwd command will print your home directory on screen, pwd means print working directory.

/u0/ssb/sandeep is output for the command when I use pwd in /u0/ssb/sandeep directory.

❖ **Ls** command

ls command is most widely used command and it displays the contents of directory.

options

- ls will list all the files in your home directory, this command has many options.
- ls -l will list all the file names, permissions, group, etc in long format.
- ls -a will list all the files including hidden files that start with .

❖ **Mkdir** command.

mkdir balaji will create new directory, i.e. here balaji directory is created.

❖ **Cd** command.

cd balaji will change directory from current directory to balaji directory.
Use pwd to check your current directory and ls to see if balaji directory is there or not.
You can then use cd balaji to change the directory to this new directory.

❖ **Cat** command

cat cal.txt cat command displays the contents of a file here **cal.txt** on screen (or standard out).

❖ **Head** command.

head filename by default will display the first 10 lines of a file.

If you want first 50 lines you can use head -50 filename or for 37 lines head -37 filename and so forth.

❖ **Tail** command.

tail filename by default will display the last 10 lines of a file.

If you want last 50 lines then you can use tail -50 filename.

❖ **More** command. more command will display a page at a time and then wait for input which is spacebar. For example if you have a file which is 500 lines and you want to read it all.

❖ **Wc** command

wc command counts the characters, words or lines in a file depending upon the option.

Options

- `wc -l filename` will print total number of lines in a file.
- `wc -w filename` will print total number of words in a file.
- `wc -c filename` will print total number of characters in a file.

❖ **File** command.

File command displays about the contents of a given file, whether it is a text (Ascii) or binary file. To use it type `file filename`.

For example I have `cal.txt` which has ascii characters about calander of current month and I have `resume1.doc` file which is a binary file in microsoft word. I will get

```
file resume.doc
      resume1.doc:  data
      file cal.txt
      cal.txt:      ascii text
```

❖ **Cp** command.

`cp` command copies a file. If I want to copy a file named `oldfile` in a current directory to a file named `newfile` in a current directory.

```
cp oldfile newfile
```

❖ **Rcp** command.

`rcp` command will copy files between two unix systems and works just like `cp` command (`-p` and `-i` options too).

For example you are on a unix system that is called *Cheetah* and want to copy a file which is in current directory to a system that is called *lion* in `/usr/john/` directory then you can use `rcp` command

```
rcp filename lion:/usr/john
```

❖ **Mv** command.

`mv` command is used to move a file from one directory to another directory or to rename a file.

Some examples:

- `mv oldfile newfile` will rename `oldfile` to `newfile`.
- `mv -i oldfile newfile` for confirmation prompt.
- `mv -f oldfile newfile` will force the rename even if target file exists.
- `mv * /usr/bajwa/` will move all the files in current directory to `/usr/bajwa` directory.

❖ **Ln** command.

Instead of copying you can also make links to existing files using `ln` command. If you want to create a link to a file called `coolfile` in `/usr/local/bin` directory then you can enter this command.

```
ln mycoolfile /usr/local/bin/coolfile
```

Some examples:

- `ln -s fileone filetwo` will create a symbolic link and can exist across machines.
- `ln -n` option will not overwrite existing files.

❖ **Rm** command.

To delete files use rm command.

Options:

- rm oldfile will delete file named oldfile.
- rm -f option will remove write-protected files without prompting.
- rm -r option will delete the entire directory as well as all the subdirectories, very dangerous command.

❖ **Rmdir** command.

rmdir command will remove directory or directories if a directory is empty.

Options:

- rm -r directory_name will remove all files even if directory is not empty.
- rmdir sandeep is how you use it to remove sandeep directory.
- rmdir -p will remove directories and any parent directories that are empty.
- rmdir -s will suppress standard error messages caused by -p.

Comparison and Searching

❖ **Diff** command.

diff command will compare the two files and print out the differences between.
Here I have two ascii text files. File one and file two.

Contents of file one are

This is first file
this is second line
this is third line
this is different as;lkdjf
this is not different

file two contains

This is first file
this is second line
this is third line
this is different xxxxxxxxas;lkdjf
this is not different

diff fileone filetwo will give following output

```
4c4
< this is different as;lkdjf
```

❖ **Cmp** command.

cmp command compares the two files. For example I have two different files fileone and filetwo.
cmp fileone filetwo will give me fileone filetwo differ: char 80, line 4

if I run cmp command on similar files nothing is returned.

-s command can be used to return exit codes. i.e. return 0 if files are identical, 1 if files are different, 2 if files are inaccessible.

This following command prints a message 'no changes' if files are same
cmp -s fileone file1 && echo 'no changes' .
no changes

❖ **Dircmp** Command.

dircmp command compares two directories. If i have two directories in my home directory named *dirone* and *dirtwo* and each has 5-10 files in it. Then

dircmp dirone dirtwo will return this

Dec 9 16:06 1997 dirone only and dirtwo only Page 1

./cal.txt	./fourth.txt
./dohazaar.txt	./rmt.txt
./four.txt	./te.txt

❖ **Grep** Command

grep command is the most useful search command. You can use it to find processes running on system, to find a pattern in a file, etc. It can be used to search one or more files to match an expression. It can also be used in conjunction with other commands as in this following example, output of ps command is passed to grep command, here it means search all processes in system and find the pattern **sleep**.

ps -ef | grep sleep will display all the sleep processes running in the system as follows.

```
ops 12964 25853 0 16:12:24 ttyAE/AAES 0:00 sleep 60
dxi 12974 15640 0 16:12:25 ttyAH/AAHP 0:00 sleep 60
ops 12941 25688 0 16:12:21 ttyAE/AAEt 0:00 sleep 60
ops 12847 25812 0 16:11:59 ttyAH/AAH6 0:00 sleep 60
```

Options:

- -b option will precede each line with its block number.
- -c option will only print the count of matched lines.
- -i ignores uppercase and lowercase distinctions.
- -l lists filenames but not matched lines.

other associated commands with **grep** are **egrep** and **fgrep**. egrep typically runs faster. for more information type man egrep or man fgrep in your system.

❖ **Find** command.

Find command is a extremely useful command. you can search for any file anywhere using this command provided that file and directory you are searching has read write attributes set to you ,your, group or all. Find descends directory tree beginning at each pathname and finds the files that meet the specified conditions. Here are some examples.

Some Examples:

find \$HOME -print will lists all files in your home directory.

find /work -name chapter1 -print will list all files named chapter1 in /work directory.

find / -type d -name 'man*' -print will list all manpage directories.

find / -size 0 -ok rm {} \; will remove all empty files on system.

conditions of find

- **-atime +n |-n| n** will find files that were last accessed more than n or less than -n days or n days.
- **-ctime +n or -n** will find that were changed +n -n or n days ago.
- **-depth** descend the directory structure, working on actual files first and then directories. You can use it with cpio command.
- **-exec command {} \;** run the Unix command on each file matched by find. Very useful condition.
- **-print** print or list to standard output (screen).
- **-name pattern** find the pattern.
- **-perm nnn** find files whose permission flags match octal number nnn.
- **-size n** find files that contain n blocks.
- **-type c** Find file whose type is c. C could be **b** or block, **c** Character special file, **d** directory, **p** fifo or named pipe, **l** symbolic link, or **f** plain file.

Text processing

❖ **Cut** command.

cut command selects a list of columns or fields from one or more files.
Option -c is for columns and -f for fields. It is entered as

cut *options* [files]

for example if a file named testfile contains

this is firstline

this is secondline

this is thirdline

Examples:

cut -c1,4 testfile will print this to standard output (screen)

ts

ts

ts

It is printing columns 1 and 4 of this file which contains t and s (part of *this*).

Options:

- **-c list** cut the column positions identified in list.
- **-f list** will cut the fields identified in list.
- **-s** could be used with -f to suppress lines without delimiters.

❖ **Paste** Command.

paste command merge the lines of one or more files into vertical columns separated by a tab.

for example if a file named testfile contains

this is firstline

and a file named testfile2 contains

this is testfile2
then running this command
paste testfile testfile2 > outputfile
will put this into outputfile
this is firstline this is testfile2
it contains contents of both files in columns.

who | paste - - will list users in two columns.

Options:

- -d'char' separate columns with char instead of a tab.
- -s merge subsequent lines from one file.

❖ **Sort** command.

sort command sort the lines of a file or files, in alphabetical order. for example if you have a file named testfile with these contents

zzz

aaa

1234

yuer

wer

qww

wwe

Then running

sort testfile

will give us output of

1234

aaa

qww

wer

wwe

yuer

zzz

Options:

- -b ignores leading spaces and tabs.
- -c checks whether files are already sorted.
- -d ignores punctuation.
- -i ignores non-printing characters.
- -n sorts in arithmetic order.
- -o file put output in a file.
- +m[-m] skips n fields before sorting, and sort upto field position m.
- -r reverse the order of sort.
- -u identical lines in input file appear only one time in output.

❖ **Uniq** command.

uniq command removes duplicate adjacent lines from sorted file while sending one copy of each second file.

Examples

sort names | uniq -d will show which lines appear more than once in names file.

Options:

- -c print each line once, counting instances of each.
- -d print duplicate lines once, but no unique lines.
- -u print only unique lines.

❖ **Awk** and **Nawk** command.

awk is more like a scripting language builtin on all unix systems. Although mostly used for text processing, etc.

Here are some examples which are connected with other commands.

Examples:

df -t | awk 'BEGIN {tot=0} \$2 == "total" {tot=tot+\$1} END {print (tot*512)/1000000}' Will give total space in your system in megabytes.

Here the output of command df -t is being passed into awk which is counting the field 1 after pattern "total" appears. Same way if you change \$1 to \$4 it will accumulate and display the addition of field 4 which is used space.

for more information about awk and nawk command in your system enter man awk or man nawk.

❖ **Sed** command.

sed command launches a stream line editor which you can use at command line. you can enter your sed commands in a file and then using -f option edit your text file. It works as sed [options] files

options:

- -e 'instruction' Apply the editing instruction to the files.
- -f *script* Apply the set of instructions from the editing script.
- -n suppress default output.

for more information about sed, enter man sed at command line in your system.

❖ **Vi** editor.

vi command launches a vi sual editor. To edit a file type

vi filename

vi editor is a default editor of all Unix systems. It has several modes. In order to write characters you will need to hit **i** to be in insert mode and then start typing. Make sure that your terminal has correct settings, vt100 emulation works good if you are logged in using pc. Once you are done typing then to be in command mode where you can write/search/ you need to hit **:w filename** to write

and in case you are done writing and want to exit

:w! will write and exit.

options:

- **i** for insert mode.
- **I** inserts text at the cursor
- **A** appends text at the end of the line.
- **a** appends text after cursor.
- **O** open a new line of text above the cursor.
- **o** open a new line of text below the cursor.
- **:** for command mode.
- **<escape>** to invoke command mode from insert mode.
- **!sh** to run unix commands.
- **x** to delete a single character.
- **dd** to delete an entire line
- **n dd** to delete n number of lines.
- **d\$** to delete from cursor to end of line.
- **yy** to copy a line to buffer.
- **P** to paste text from buffer.
- **nyy** copy n number of lines to buffer.
- **:%s/stringA/stringb /g** to replace stringA with stringB in whole file.
- **G** to go to last line in file.
- **1G** to go to the first line in file.
- **w** to move forward to next word.
- **b** to move backwards to next word.
- **\$** to move to the end of line.
- **J** join a line with the one below it.
- **/string** to search string in file.
- **n** to search for next occurrence of string.

SQL Queries

1. DUPLICATE QUERIES:

Count Duplicate Records:

Select CN from TN group by CN having count(CN)>1;

Display Duplicate Records:

- Select * from TN A where Rowid>(Select min(Rowid) from TN B where A.CN=B.CN);
- Select * from TN where Rowid Not IN(Select min(rowid) from TN group by CN);

Display Original Records:

Select * from TN A where Rowid=(Select min(rowid) from TN B where A.CN=B.CN);

Delete Duplicate Records original exists as it is:

Delete from TN A Where A.rowid>(select min(Rowid) from TN B where A.Id=B.Id);

All Original Records:

Select * from TN where Rowid In(Select min(rowid) from TN group by Id);

2.1st RECORDS:

- Select * from TN where Rowid=(select min(Rowid) from TN);
- Select * from TN where rownum=1;
- Select * from (select TN.*, Rownum() over (order by rownum) RN from TN) where RN=1;
- Select * from (select TN.*, Rownum RN from TN) where RN=1
- Select * from TN Minus (Select * from TN where Rowid IN(Select rowid from TN where Rowid Not In(select max(Rowid) from TN)));

3. Last Records:

- Select * from TN where rowid=(select max(Rowid) from TN);
- Select * from(select TN.*, Rownum()over (order by Rownum Desc) RN from TN) where RN=1;
- Select *from TN Minus (select * from TN where Rowid in(select Rowid from TN where Rowid not In(select max(Rowid) from TN)));

4. Middle Records:

- Select * from TN where Rowid <=(select Rowid(count(*)/2) from TN MINUS Select * from TN where rownum<>(select Rowid(count (*)/2) from TN);
- Select * from TN where Rownum<=(select Rowid(max (Rownum)/2) from TN MINUS select * form TN where Rownum <(select Rowid(max(Rownum)/2) from TN);
- Select * from (select TN.*, Rownum RN from TN) where RN=(select Rowid(count (*)/2) from TN);

5. Nth Records:

- Select * from(select TN.*, Dense_rank() over(order by Rownum) RN where RN=&RN;
- Select * from(select TN.*, Row_Number() over(order by Rownum) RN where RN=&RN;
- Select * from TN where Rownum >3 MINUS Select * from TN where Rownum <8;
- Select * from(select TN.*, Rownum RN from TN) where RN<11 and RN>9;
- Select * from(select TN.* Rownum RN from TN) where RN=9 or RN=11;

6. Least Nth Records:

- Select * from(select TN.*, Dense_rank() over (order by Rownum) RN from TN) where RN =(select count(*)-&R+1 from TN);
- Select * from(select TN.*, Rownum RN from TN) where RN >=(select count(*)-&R+1 from TN);
- Select * from(select * from TN order by Rowid Desc) where rownum<=10;

7. Top Nth Records:

- Select * from TN where Rownum <=10;
- Select * from TN where Rownum <=&D;

8. Display original and Duplicate Records:

- Select * from TN where Id In(select Id from TN group by Id having count(id)>1;
- Select * from TN A where exists(select * from TN B where A.Id=B.Id and A.rowid<>B.Rowid;

Even Records:

Select * from emp where (rowid,0) in select rowid,mod(rowid,2) from emp;

odd Records:

Select * from emp where (rowid,1) in select rowid,mod(rowid,2) from emp;

Level Function:

Select substr('LOKESH' 1, Level) from Dual connect by level<= Length ('AMOL');

1) Display employees having salary is equal to 3897.43.

select * from employees where salary=3897.43;

2) Display employees having salary less than 4000 and hiring year greater than 1990.

select * from employees where salary < 4000 and to_char(HIRE_DATE,'YYYY')>1990;

3) Display all employees in IT or Accounting department.

it will display all details

select * from employees inner join departments on employees.department_id=departments.department_id
where departments.department_name in ('IT','Accounting');

it will display only emp_id,first name, dep name

```
select E.EMPLOYEE_ID,e.first_name,d.department_name from employees e inner join departments d on e.department_id=d.department_id where d.department_name in ('IT','Accounting');
```

4) How to display employees whose salary between 2500 and 5000 without using between operators?

```
select * from employees where salary>=2500 and salary<=5000;
```

5) Display employees whose COMMISSION_PCT is other than 0.10.

```
select * from employees where COMMISSION_PCT!=0.10;
```

```
select * from employees where COMMISSION_PCT not in 0.10;
```

```
select first_name, last_name,commission_pct from employees where COMMISSION_PCT!=0.10;
```

6) Delete employees from department Purchasing and salary less than 2500.

```
delete from employees E where employee_id in (select e.employee_id from employees E inner join departments d on e.department_id=d.department_id where d.department_name = 'Purchasing' and e.salary < 50000);
```

7) Display first name last name and salary of an employee who is having max salary.

```
select first_name, last_name, salary from employees where salary=(select max(salary)from employees);
```

8) What is the Query to fetch last record from the employees table?

--1st Method

```
select employees.* from employees where rowid = (select max(rowid) from employees);
```

--2nd Method

```
select * from (select employees.*,rownum r from employees) where r=(select max(rownum) from employees);
```

9) What is the Query to fetch first record from Employee table?

--1st approach

```
select * from employees where rowid = (select min (rowid) from employees);
```

--2nd approach

```
select * from employees where rownum = 1;
```

10) What is Query to display last 5 Records from Employee table?

--1st approach

```
select * from (select employees.*, dense_rank() over (order by (rownum) desc) as drank from employees) where drank between 1 and 5;
```

--2nd approach

```
select * from (select rownum as extrarow, employees.* from employees) where extrarow > (select count(*)-5 from employees);
```

--3rd approach

```
select * from (select employees.* from employees order by rownum desc) where rownum<=5
```

11) Display Odd rows in Employee table

-- 1st Method using rank

```
select * from (select rownum, employees.*, dense_rank() over (order by (rownum)) as drank from employees) where mod(drank,2)<>0;
```

-- 2nd method using rownum

```
select * from (select rownum as seq, employees.* from employees) where mod(seq,2)=1;
```

12) Display Even rows in Employee table

-- 1st Method using rank

```
select * from (select rownum, employees.*, dense_rank() over (order by (rownum)) as drank from employees) where mod(drank,2)=0;
```

-- 2nd method using rownum

```
select * from (select rownum as seq, employees.* from employees) where mod(seq,2)=0;
```

13) Display total count of Employees.

```
select count (employee_id) from employees;
```

14) Display department Wise total count of Employees.

```
select d.department_name,count(d.department_name) from employees e
inner join departments d on e.department_id=d.department_id
group by d.department_name;
```

15) Display region Wise total count of Employees.

```
select r.region_name,count(E.employee_id)from employees E
inner join departments D on e.department_id = d.department_id
inner join locations l on d.location_id=l.location_id
inner join countries C on c.country_id=L.COUNTRY_ID
inner join regions R on C.region_id = R.region_id
group by r.region_name;
```

16) Display the list of employees having first name 'David' or 'Diana' without using like, in and or operators.

-- 1st Approach using any

```
select * from employees where first_name = any ('David','Diana');
```

--2nd approach

```
select * from employees where first_name = 'David'
```

```
union all
```

```
select * from employees where first_name = 'Diana';
```

17) Display employee id, employee full name in one column and his/her Manager id Manager full name in another column.

--e.g. |Employeeid-Name||Managerid-Name|

-- error in question - manager name is not present in any of the table

```
select employee_ID||'-'||first_name||'-'||last_name as firstset,manager_id||'-'||first_name as secondset from employees
```

18) Display department name having sum of employees salary is greater than 25000 and employees count is less than 10

1st way

```
select * from (
```

```
select
```

```
d.department_name,sum(e.salary) as sum_of_sal,count(e.employee_id) Count_Of_Emp
```

```
from departments D join employees e
```

```
on d.department_id=e.department_id
```

```
where e.salary>2500
```

```
group by
```

```
d.department_name
```

```
)
```

```
where Count_Of_Emp<10;
```

--2nd way by using having

```
select
```

```
d.department_name,sum(e.salary),count(e.employee_id)
```

```
from departments D join employees e
```

```
on d.department_id=e.department_id
```

```
where e.salary>2500
```

group by
d.department_name
having
count(e.employee_id)<10;

19) Write SQL query to update all employees' salary by 10%.

update employees
set salary = (salary + (salary * 0.1));

20) Write SQL query to update employees' salary by 15% and Region of employees should be Asia.

--1st Approach

update employees E set salary = salary * 1.10 where e.employee_id in
(select E.employee_id from employees E
inner join departments D on e.department_id = d.department_id
inner join locations l on d.location_id=l.location_id
inner join countries C on c.country_id=L.COUNTRY_ID
inner join regions R on C.region_id = R.region_id
where R.REGION_NAME = 'Europe');

--2nd Approach

update
(select E.salary,r.region_name from employees E
inner join departments D on e.department_id = d.department_id
inner join locations l on d.location_id=l.location_id
inner join countries C on c.country_id=L.COUNTRY_ID
inner join regions R on C.region_id = R.region_id
where R.REGION_NAME = 'Europe')T
set T.salary=(T.salary)*1.10;
rollback;

21) Write SQL query to create the EMPLOYEES_1 table, which is exact replica of EMPLOYEES table create table employees_1 as (select * from employees);

```
select * from employees_1
```

22) Write SQL query to find current experience of employees.

```
select first_name,last_name,hire_date,  
round(sysdate-hire_date,2) as cu_exp_days,  
round(months_between (SYSDATE,HIRE_DATE),2) as cu_exp_month,  
round(months_between (SYSDATE,HIRE_DATE)/12,2) as cu_exp_year from employees;
```

23) Write a query to display list of employees having salary greater than Max salary define for respective JOB_ID.

-- logic - join the table employees and jobs w.r.t job_id and display the table only if salary > max(salary) column

```
select e.first_name,e.job_id,salary,J.MIN_SALARY,J.MAX_SALARY from EMPLOYEES E  
inner join jobs j on E.JOB_ID=J.JOB_ID where e.salary>j.max_Salary;  
and e.job_id='AD_VP' order by job_id; -- add if required to find for individual dept  
-- and in where is used to display in simple way for individual dept
```

24) Write a query to count number of rows in table without using count function?

```
select count(*) from employees; -- for checking  
select max(rownum) from employees;
```

25) Write a query to display department-wise max salary. Output columnsà EMP_ID, FIRST_NAME, SALARY, DEPARTMENT

```
select * from (select E.employee_id,E.FIRST_NAME,E.salary,D.DEPARTMENT_NAME,  
dense_rank() over (partition by d.department_name order by salary) as drank  
from employees E  
inner join departments D on e.department_id = d.department_id) where drank = 1
```

26) Write a query to display department-wise 2nd max salary. Output columns EMP_ID, FIRST_NAME, SALARY, DEPARTMENT

--2nd max salary

-- this query gives wrong output

```
select E.employee_id,E.FIRST_NAME,E.Last_name, E.salary,D.DEPARTMENT_NAME from employees E
```

inner join departments D on e.department_id = d.department_id where salary

in (select max(salary) from employees where salary < (select max(salary) from employees) group by department_id) order by department_name desc;

-- correct query

select * from (select employees.*, dense_rank() over (partition by department_id order by salary desc) as derank from employees) where derank=2

-- department wise 2nd max salary with department name using join and drank

select * from (select dense_rank() over (partition by e.department_id order by (salary) desc) as drank, e.salary, e.first_name, e.department_id, d.department_name from employees e inner join departments d on e.department_id = d.department_id) where drank = 1;

---wrong one below

select max(Salary), max(employee_id), max(first_name), department_name from employees, departments where employees.department_id=departments.department_id group by department_name;

--- this returns all the details such as city, country and region using drank and join

select * from (
select dense_rank() over (partition by e.department_id order by (salary) desc) as drank,
e.salary, e.first_name, e.department_id, d.department_name, l.city, c.country_name, r.region_name from employees e
inner join departments d on e.department_id = d.department_id
inner join locations l on d.location_id = l.location_id
inner join countries C on l.country_id=c.country_id
inner join regions R on c.region_id=r.region_id
) where drank = 2;

27) Write a query to display EMPLOYEES having ID 101,102,103 as per below order:

-- 1. 101-- 2. 103-- 3. 102

select employees.* from employees where employee_id > 100 and employee_id < 104
order by
case
when employee_id = 101 then 1
when employee_id = 103 then 2
when employee_id = 102 then 3

when first_name = 'Neena' then 4

else 5

end;

-- 2nd approach to display

Select * from EMPLOYEES where employee_ID = 101 Union All

Select * from employees where employee_ID = 103 Union all

select * from employees where employee_id = 110 union all

select * from employees where first_name = 'Neena' ;

-- extra work

select * from employees where first_name = 'Jennifer';

select * from employees, job_history;

Select * from employees group by department_id sort by employee_id;

select first_name from employees where Department_id = 100;

select count(*), department_id from employees group by department_id;

select max(salary), first_name, department_id from employees group by first_name,department_id;

select sum(salary), department_id from employees group by department_id;

select avg(salary), department_id from employees group by department_id;

select max(salary)as jyadasal, department_id from employees group by department_id;

select first_name, max(salary)as jyadasal, department_id from employees group by department_id,
first_name having first_name = 'Neena';

select department_id, count(salary)as sal,sum(department_id)as sum from employees group by
department_id;

select department_id, max(salary)as sal from employees group by department_id;

select first_name, department_id, count(first_name),sum(salary) from employees group by
department_id,first_name order by department_id asc;

select * from employees order by salary desc;

select * from employees order by salary asc;

select * from employees order by job_id asc,salary desc; -- nulls are considered as highest

select * from employees order by manager_id desc; -- here null comes first

select * from employees order by manager_id desc nulls last; -- to display null value at last

Challenges faced during ETL

- Data loss during the ETL process.
- Incorrect, incomplete or duplicate data.
- DW system contains historical data, so the data volume is too large and extremely complex to perform ETL testing in the target system.
- ETL testers are normally not provided with access to see job schedules in the ETL tool. They hardly have access to BI Reporting tools to see the final layout of reports and data inside the reports.
- Tough to generate and build test cases, as data volume is too high and complex.
- ETL testers normally don't have an idea of end-user report requirements and business flow of the information.
- ETL testing involves various complex SQL concepts for data validation in the target system.
- Sometimes the testers are not provided with the source-to-target mapping information.

ETL Testing – Tasks to be Performed

- Understand the data to be used for reporting
- Review the Data Model
- Source to target mapping
- Data checks on source data
- Packages and schema validation
- Data verification in the target system
- Verification of data transformation calculations and aggregation rules
- Sample data comparison between the source and the target system
- Data integrity and quality checks in the target system
- Performance testing on data

Roles of ETL Tester

Verify the Tables in the Source System

It involves the following operations –

- Count check
- Reconcile records with the source data
- Data type check
- Ensure no spam data loaded
- Remove duplicate data
- Check all the keys are in place

Apply Transformation Logic

Transformation logic is applied before loading the data. It involves the following operations –

- Data threshold validation check, for example, age value shouldn't be more than 100.
- Record count check, before and after the transformation logic applied.
- Data flow validation from the staging area to the intermediate tables.
- Surrogate key check.

Data Loading

Data is loaded from the staging area to the target system. It involves the following operations –

- Record count check from the intermediate table to the target system.
- Ensure the key field data is not missing or Null.
- Check if the aggregate values and calculated measures are loaded in the fact tables.
- Check modeling views based on the target tables.
- Check if CDC has been applied on the incremental load table.
- Data check in dimension table and history table check.
- Check the BI reports based on the loaded fact and dimension table and as per the expected results.

ETL Testing Process

- Understanding the business requirement.
- Validation of the business requirement.
- Test Estimation is used to provide the estimated time to run test-cases and to complete the summary report.
- Test Planning involves finding the Testing technique based on the inputs as per business

ETL Test Scenarios are used to validate an ETL Testing Process

Test Scenarios	Test-Cases
Structure Validation	It involves validating the source and the target table structure as per the mapping document. Data type should be validated in the source and the target systems. The length of data types in the source and the target system should be same. Data field types and their format should be same in the source and the target system. Validating the column names in the target system.
Validating Mapping document	It involves validating the mapping document to ensure all the information has been provided. The mapping document should have change log, maintain data types, length, transformation rules, etc.
Validate Constraints	It involves validating the constraints and ensuring that they are applied on the expected tables.
Data Consistency check	It involves checking the misuse of integrity constraints like Foreign Key. The length and data type of an attribute may vary in different tables, though their definition remains same at the semantic layer.
Data Completeness Validation	It involves checking if all the data is loaded to the target system from the source system. Counting the number of records in the source and the target systems. Boundary value analysis. Validating the unique values of primary keys.
Data Correctness Validation	It involves validating the values of data in the target system. Misspelled or inaccurate data is found in table. Null, Not Unique data is stored when you disable integrity constraint at the time of import.
Data Transform validation	It involves creating a spreadsheet of scenarios for input values and expected results and then validating with end-users. Validating parent-child relationship in the data by creating scenarios. Using data profiling to compare the range of values in each field. Validating if the data types in the warehouse are same as mentioned in the data model.

Data Quality Validation	It involves performing number check, date check, precision check, datacheck, Null check, etc. Example – Date format should be same for all the values.
Null Validation	It involves checking the Null values where Not Null is mentioned for that field.
Duplicate Validation	It involves validating duplicate values in the target system when data is coming from multiple columns from the source system. Validating primary keys and other columns if there is any duplicate values as per the business requirement.
Date Validation check	Validating date field for various actions performed in ETL process. Common test-cases to perform Date validation – • From_Date should not greater than To_Date • Format of date values should be proper. • Date values should not have any junk values or null values
Full Data Validation Minus Query	It involves validating full data set in the source and the target tables by using minus query. • You need to perform both source minus target and target minus source. • If the minus query returns a value, that should be considered as mismatching rows. • You need to match the rows in source and target using the Intersect statement. • The count returned by Intersect should match with the individual counts of source and target tables. • If the minus query returns no rows and the count intersect is less than the source count or the target table count, then the table holds duplicate rows.
Other Test Scenarios	Other Test scenarios can be to verify that the extraction process did not extract duplicate data from the source system. The testing team will maintain a list of SQL statements that are run to validate that no duplicate data have been extracted from the source systems.
Data Cleaning	Unwanted data should be removed before loading the data to the staging area.